

# TÍTULO - "*ROOTLETS*"

**Informe del Proyecto: Proyecto  
de desarrollo de aplicaciones  
multiplataforma / web**

**CURSO 2023/24**

Nombre y apellidos

**CENEC**  
MALAGA



# ÍNDICE

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1.- Descripción</b>                                             | <b>2</b>  |
| <b>2.- Análisis del Mercado Actual</b>                             | <b>3</b>  |
| HubSpot                                                            | 3         |
| Monday.com                                                         | 3         |
| Freshsales                                                         | 4         |
| <b>3.- Propuesta Innovadora</b>                                    | <b>4</b>  |
| <b>4.- Estudio de Viabilidad</b>                                   | <b>5</b>  |
| 4.1.- Descripción del modelo de negocio                            | 5         |
| 4.2.- Embudo de marketing                                          | 5         |
| 4.3.- Estimación de la población objetivo                          | 6         |
| 4.4.- Cálculo de la muestra poblacional necesaria para la encuesta | 6         |
| 4.5.- Encuesta y resultados                                        | 6         |
| 4.6.- Planificación del tiempo                                     | 11        |
| 4.7.- Estimación de costes                                         | 12        |
| 4.8.- Estimación de ingresos                                       | 12        |
| 4.9.- Conclusiones                                                 | 13        |
| <b>5.- Arquitectura de la Aplicación</b>                           | <b>13</b> |
| 5.1.- Recursos y tecnologías empleadas                             | 13        |
| 5.2.- Diseño inicial                                               | 13        |
| 5.3.- Diseño final                                                 | 20        |
| <b>6.- Futuras Mejoras y Conclusiones</b>                          | <b>26</b> |
| <b>7.- Pruebas y Resultados</b>                                    | <b>27</b> |
| <b>8.- Manual Usuario</b>                                          | <b>30</b> |
| <b>9.- Bibliografía</b>                                            | <b>36</b> |
| <b>10 - Anexo I.- Principales clases del código</b>                | <b>37</b> |
| <b>11 - Anexo 2: Opinión personal sobre la FCT</b>                 | <b>60</b> |

# 1.- Descripción

Customer Relationship Management (Gestión de la relaciones con clientes) es una estrategia ampliamente implementada para la interacción de una empresa administradora con clientes y prospectos de ventas. Su objeto es para organizar, automatizar y sincronizar las actividades de ventas y el proceso de negocios, pero también para servicios al cliente y marketing. Cualquier persona puede usar este sistema ya que es virtual y no requiere oficinas o entornos físicos. El propietario puede comunicarse y proporcionar servicios a través de este sistema y los clientes pueden acceder y rastrear su estado, así como más detalles de sus proyectos.

Hay muchos sistemas CRM en el mercado, pero ninguno tiene ventana para clientes. Por lo tanto, todo el trabajo se realizará de forma manual y debe anotarse y cuidar la documentación para que los clientes puedan verificar fácilmente lo que está sucediendo en su proyecto. Todo este proceso puede ayudar a evitar la falta de comunicación entre desarrolladores y clientes y construir una relación y confianza más profunda . La empresa puede informar silenciosamente al cliente sobre su estado y notificar su calendario de reuniones y también puede rastrear el pago y los detalles del proyecto. Mediante este sistema, la compañía tendrá una gran ventaja al tener registros de un año masivo y todos los datos juntos.

Mi sistema CRM se llama ROOTLETS, que significa raíces pequeñas. Cada empresa depende de muchos aspectos distintos, como clientes, empleados, servicios, etc. para llevarlos al éxito y generar la confianza entre la empresa y el cliente. Entonces, todos esos aspectos funcionarían como una pequeña raíz de la planta que representa el éxito de la compañía.

Tenemos principalmente cuatro roles, es decir tipo de usuario: Administrador, Desarrollador del negocio quien trabaja en ventas o marketing, Desarrolladores y Diseñadores y finalmente clientes. Cada uno de ellos puede acceder a cierta información y cierta funcionalidad.

## 2.- Análisis del Mercado Actual

### HUBSPOT



Está dirigida a pequeñas empresas que buscan nuevos clientes. Entre sus puntos fuertes destaca que es gratis sin limitación de número de usuarios ni restricción del tiempo. A parte de la función básica de CRM, tiene Blog para encontrar clientes potenciales. Además, el sistema verifica todas las actividades de ese cliente en el sitio web, envía un correo electrónico con más recursos y ofrece los productos o servicios. Por contra, no está hecho para grandes empresas y no tiene suficiente almacenamiento para guardar logs y actividades. por ejemplo, podrá ver solos los primeros 7 días de seguimiento de la actividad del sitio web en un contacto. Tampoco tienen capacidad para organizar procesos y proyectos muy complejos.

### MONDAY.COM



Monday.com tiene una interfaz de usuario limpia, moderna y fácil de usar. Se especializan en la organización del proyecto. Es decir, que permite al usuario personalizar y visualizar las tareas que deben completarse y actualizarlas con informes de estado y otra información relevante. Cada miembro del proyecto puede ver en qué tareas trabajan los demás miembros. eso hace que sea más fácil pasar el trabajo de una persona a otra y encontrar la opción alternativa sin retrasar el proyecto cuando una persona no está disponible de repente. Su punto débil es que tiene demasiada variedad de precios y planes. Permiten tener catorce días de prueba, lo que no es suficiente para comprender toda la funcionalidad de esta aplicación y tiene una amplia gama de precios que depende

del almacenamiento, el número de usuarios y ciertas funciones, como es la vista de timeline, vista de calendario, seguimiento de tiempo y mucho más.

## FRESHSALES



Tienen una interfaz increíblemente fácil de usar y organizan no solo el proyecto sino también el trabajo individual y la gestión del tiempo. La gran diferencia con respecto a los dos sistemas anteriores es personalizar su ventana para cada trabajador, como los detalles de contacto de las personas que trabajan actualmente, el calendario con sus tareas de muestra. Para insertar detalles de clientes, Freshsales hace la mayor parte del trabajo por el usuario. Este solo necesita introducir la URL de la empresa cliente y el sistema agrega automáticamente toda la información pública sobre la empresa, incluido el logotipo, las redes sociales, el tamaño de la empresa, la dirección, etc. También se puede sincronizar con correo electrónico. Dado que está enfocado en simplificar el trabajo del usuario en la aplicación y administrar automáticamente el calendario y los detalles de contacto por parte de inteligencia artificial, necesitan trabajar más en la seguridad y en la inteligencia artificial que captura información incorrecta de los detalles de contacto.

### 3.- Propuesta Innovadora

Hubspot se centra en encontrar nuevos clientes y clientes potenciales, por lo que se puede decir que es para una start-up. Monday.com visualiza el proceso del proyecto y se especializa en una interfaz de usuario moderna para atraer ópticamente a los usuarios. Freshsales se enfoca a la facilidad de uso, por lo que incluso las personas que no usa el ordenador frecuentemente pueden manejarla cómodamente. Todos ellos tienen diferente fuerza y tienen diferentes objetivos. Es por eso que todos tienen éxito en el mercado.

El punto que diferencia mi proyecto de esos es que permiten al cliente seguir el proceso del proyecto si lo desean. La mayoría de los clientes no saben cómo va el proyecto y los hace sentir ansiosos y preocupados, lo que genera incertidumbre y debilita la confianza. Entonces, con la función que les permite a los clientes saber dónde estamos hoy en el proyecto, puede evitarlos. Como todos sabemos, es casi imposible evitar algún

conflicto durante el proyecto entre desarrolladores y clientes debido a la falta de comunicación. El objetivo principal es facilitar la comunicación entre clientes y desarrolladores y diseñadores.

También me gustaría combinar los puntos fuertes de las tres competiciones anteriores. Proponer una interfaz simple y limpia para todos los usuarios incluyendo a alguien que no está acostumbrado a manejar una computadora a diario y también atraer nuevos clientes. Permite al usuario tener un horario personal con tareas y contacto para facilitar la organización del trabajo.

De vez en cuando, el trabajo puede ser repetitivo y frustrante, y el sistema de CRM típico a veces puede ser bastante aburrido y demasiado serio, por lo tanto, me gustaría agregar un poco de diversión para que los principiantes puedan aprender a usar esta herramienta con más motivación y que el usuario tenga una pequeña alegría en el trabajo diario.

## **4.- Estudio de Viabilidad**

### **4.1.- DESCRIPCIÓN DEL MODELO DE NEGOCIO**

Me gustaría vender mi software cuando esté perfectamente hecho. significa que la seguridad debe probarse bien, como la autenticación, la protección adicional para datos confidenciales, la configuración adecuada para el control de acceso y la seguridad, etc. Diría que no es muy realista vender mi software con mi conocimiento actual de ciberseguridad. En el futuro, cuando esté completamente preparado con suficiente comprensión de las necesidades para publicar este software, consideraría venderlo en línea. El precio dependerá de la necesidad de capacidad de almacenamiento de cada cliente, por ejemplo, el tamaño del almacenamiento de archivos, cuánto tiempo ellos quiere guardar su actividad, cuántos contactos pueden almacenar y etc. Entonces, sí, seguiría el modelo de negocio de suscripción. El plan más barato (plan básico) sería de alrededor de 20 a 30 euros por mes, y según la demanda de cada cliente, el precio sería más alto. Podría haber un plan anual para hacerlo un poco más barato. Necesitaría usar dinero no solo para publicidad sino también para dominio y base de datos. Mi software no sería sin fines de lucro con respecto al tiempo que se consumiría para crear y administrar este software. Además, creo que corregiría los errores, seguiría actualizando y mejorando las funciones y proporcionaría soporte al cliente por correo electrónico o redes sociales si hay clientes que estén dispuestos a usar. De modo que sería justo obtener alguna ganancia por mi tiempo.

## 4.2.- EMBUDO DE MARKETING

Pondría google Ads y Twitter. Ya que mi Target es una empresa, entiendo que ellos mirarían internet más que periódico.

Primera impresión:

Tendría web con colores pero simple y limpio sin complicidad para que todos los usuarios, incluso los que no usan ordenador diariamente, pueda manejar fácilmente. Colores de marca sería familia de verde que es color de la calma y tiene efecto de reducir estrés. También usaría Color amarillo que es un color positivo y activo. Logotipo puede ser un símbolo muy simple. Creo que un logo simple le da sensación de fiabilidad a la gente más que uno con muchos detalles y colores.

Como es una herramienta que se trata datos personales, demasiadas ofertas o promociones puede dar una impresión de que no es de fiar, por lo tanto, básicamente tendría precio fijo pero ofrecería una prueba gratuita de 2 semanas. Con misma razón de anterior, no daría novedades ni recompensas. Podría ofrecer la variedad de planes (número de usuarios, almacenamiento, seguridad y funciones extras) y cada cierto tiempo podría dar una promoción de probar plan distinto sin cobrar más. Podría ofrecer programa de afiliados y los usuarios pueden ganar un porcentaje.

## 4.3.- ESTIMACIÓN DE LA POBLACIÓN OBJETIVO

Las empresas, los autónomos y cualquier persona que trabaje en los proyectos con clientes lo cual requiere organización de tiempo y tareas. Es decir, las personas que tengan desde 18 años hasta edad de jubilación. No sería ideal poner más límite por rango de edad, aunque la adopción de un nuevo sistema podría ser un obstáculo para algunas personas que realmente no están acostumbradas a administrar y organizar proyectos en online, por esa razón, será necesario concentrarse en la interfaz de usuario para no parecer demasiado complejo.

Sin embargo, el número de personas en ese rango de edad es enorme y será inapropiado estimar el número de la población objetivo, por lo que pondría aquí es las personas que trabajan en el sector informático en España.

Número de empresas del sector informático es 34.004 y dando empleo a 496.761 personas.

## 4.4.- CÁLCULO DE LA MUESTRA POBLACIONAL NECESARIA PARA LA ENCUESTA

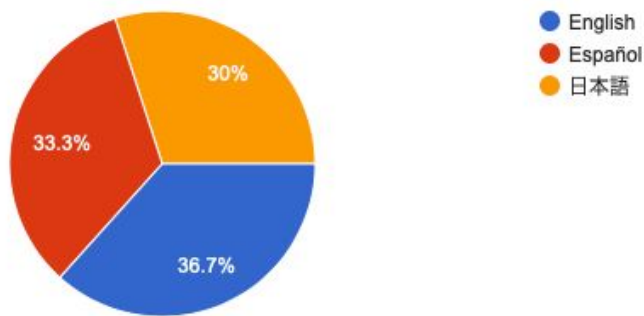
Mi objetivo son las personas que están a cargo de tomar decisiones dentro de la empresa, no los empleados. Para que la población objetivo sea 34004. Pongo un nivel de confianza del 95%, las probabilidades de respuestas favorables y las probabilidades de respuestas en contra son 0.5 como número predeterminado. Para la precisión, pongo un valor en torno al 1%. Ahora tengo suficiente información para poner en la ecuación para calcular la

muestra poblacional necesaria para la encuesta. El resultado es 95.77231969, así que el número de personas que debería entrevistar es 96.

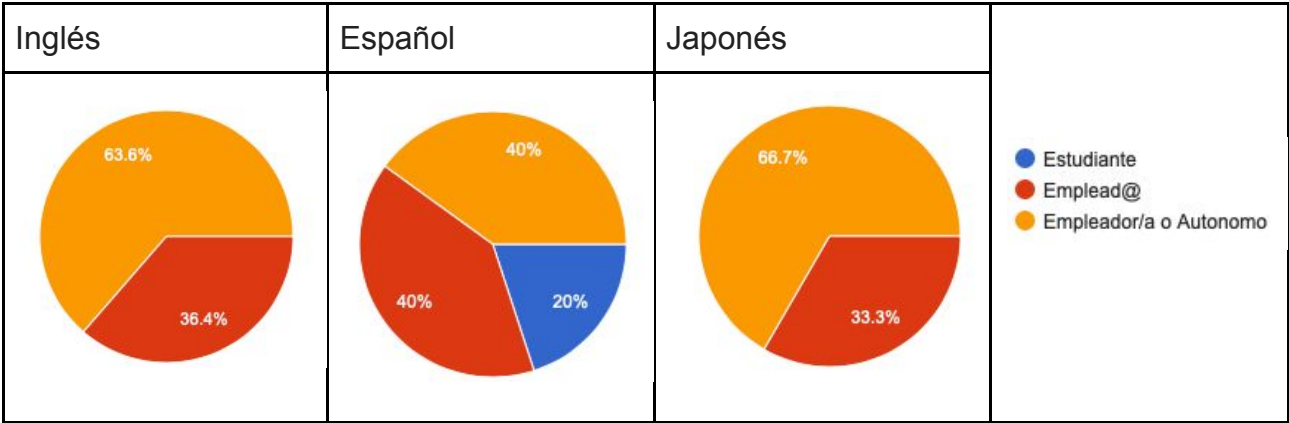
4.5.- ENCUESTA Y RESULTADOS

Hice las encuestas para que alguien prefiera en inglés, español o japonés. Entonces hay tres resultados distintas en cada pregunta.

Primero, aquí está la proporción de qué idioma eligió la gente.



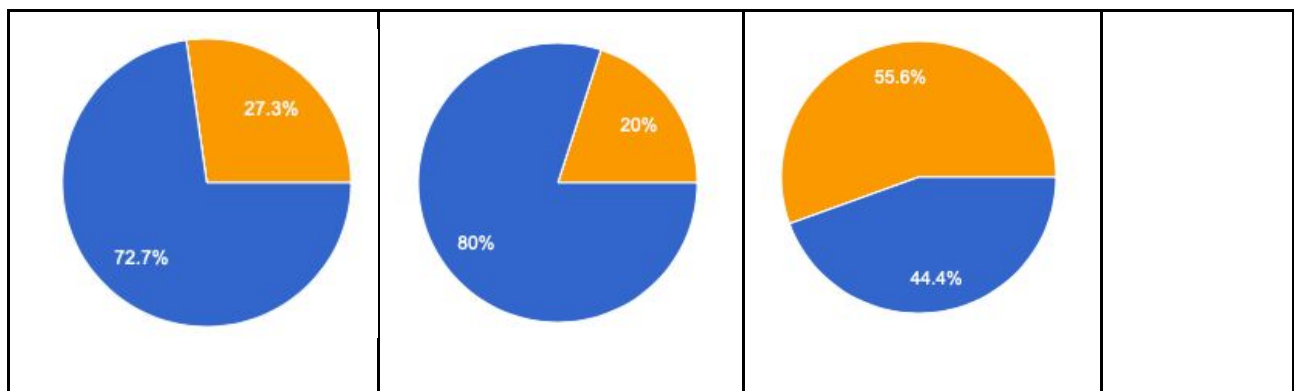
Pregunta 1: ¿Cuál es su profesión? (Las persona que se encargan de tomar la decisión dentro de una empresa elija "Empleador/a o Autónomo")



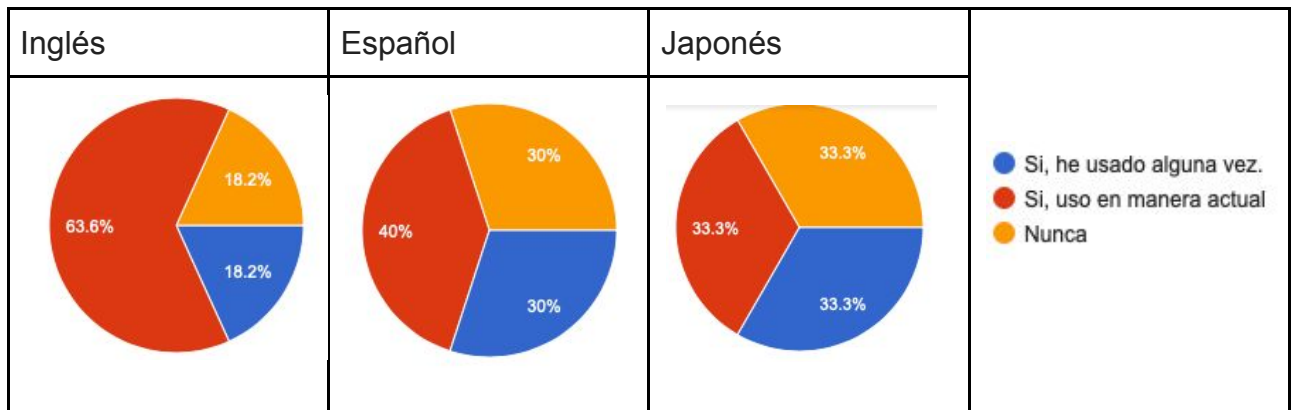
Pregunta 2: ¿Cree que herramienta CRM puede ser eficaz al negocio?





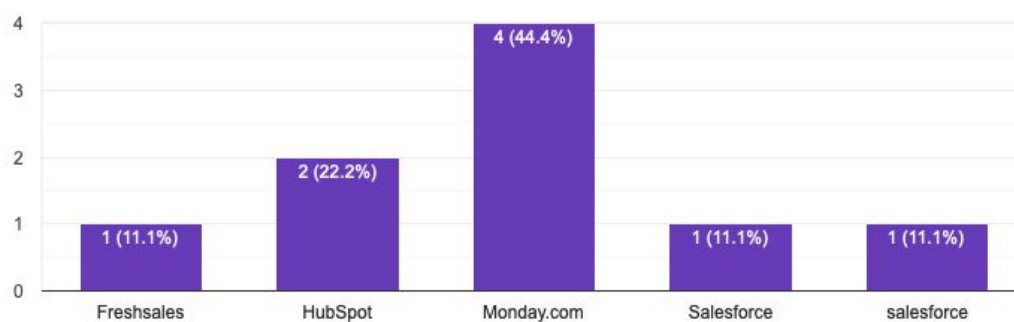


Pregunta 3: ¿Ha usado herramienta CRM alguna vez o usando en manera actual?

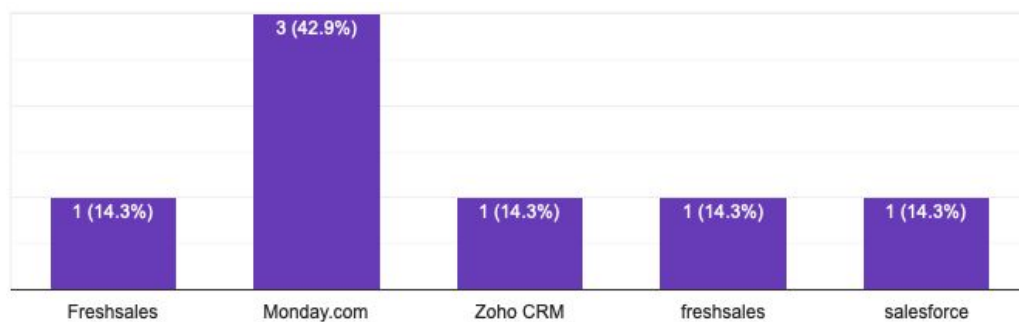


Pregunta 4: Si ha contestado "Sí" en pregunta anterior, ¿Qué herramienta ha usado / está usando?

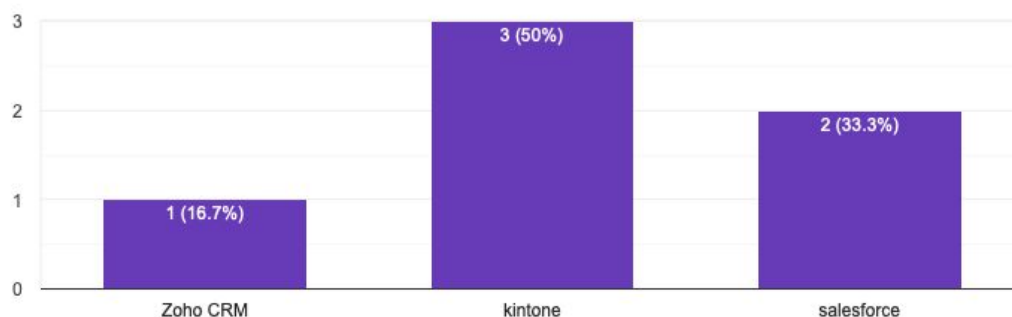
Inglés



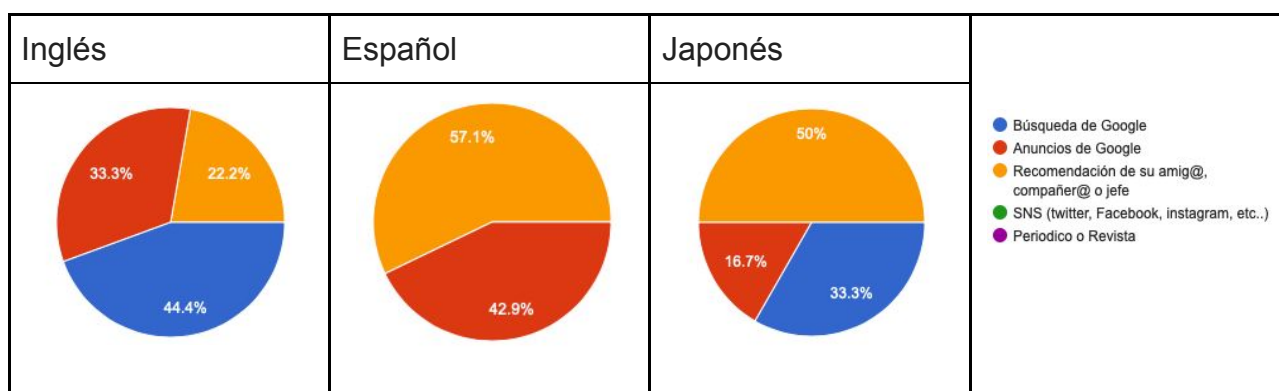
Español



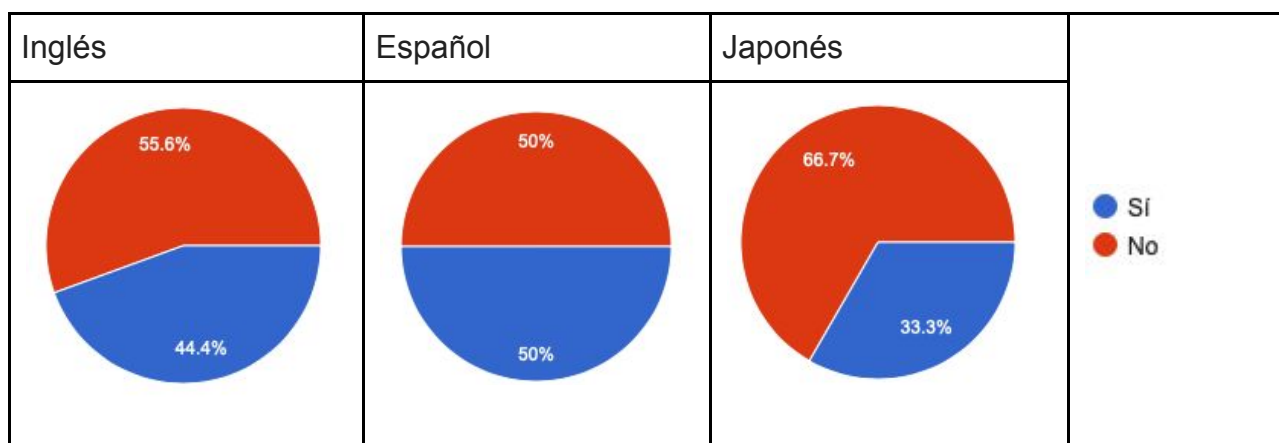
### Japonés



### Pregunta 5: ¿Cómo conoció ese software?

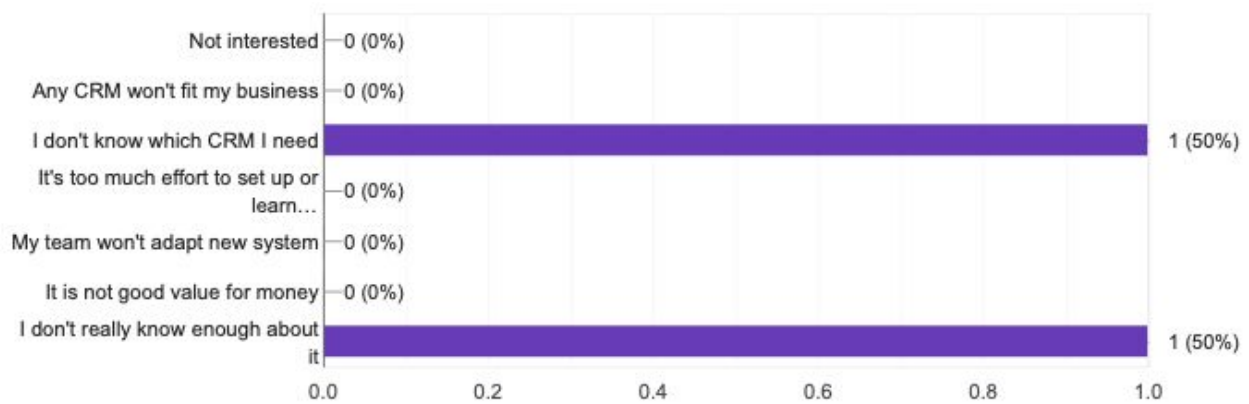


### Pregunta 6: ¿Ha recomendado a alguien?

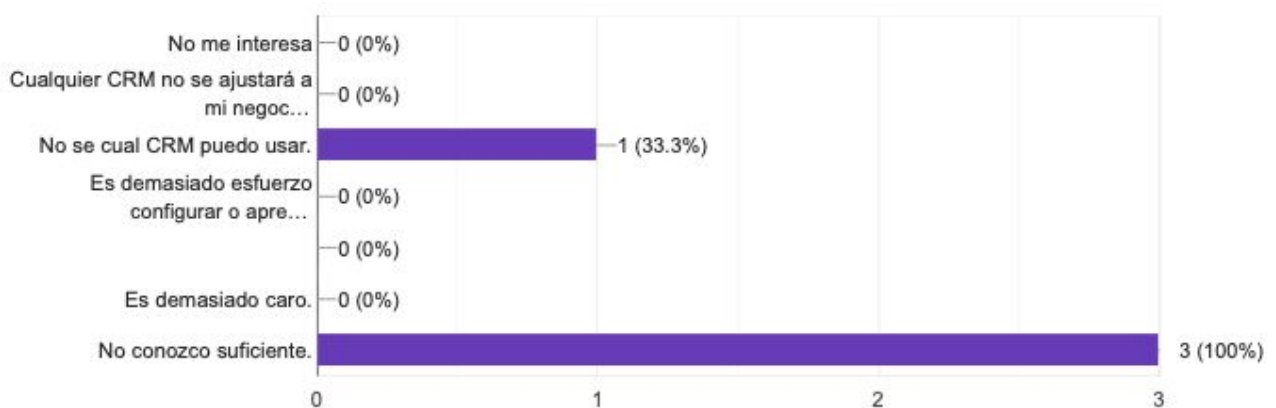


Pregunta 7: Si ha contestado "Nunca" en tercera pregunta, ¿Cuál es la razón principal de no usar?

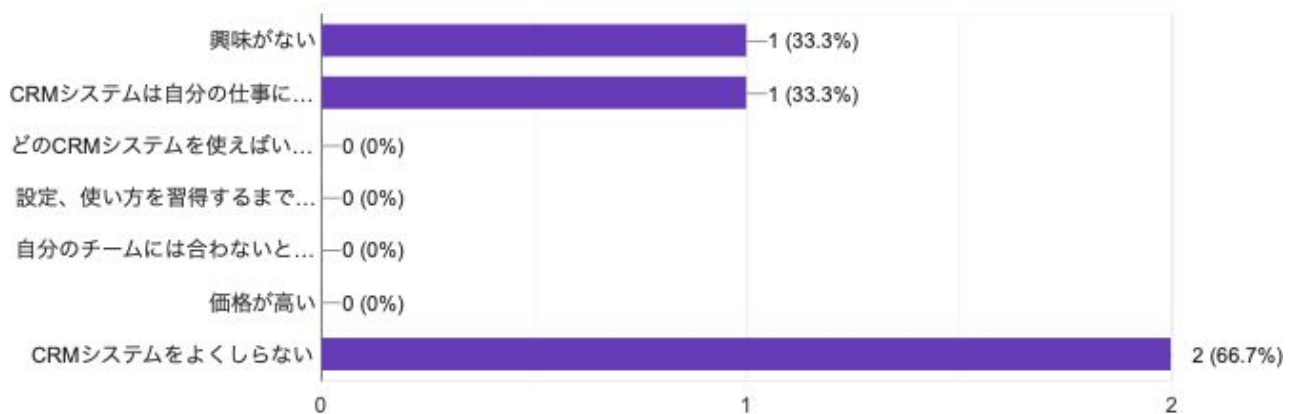
Inglés



Español

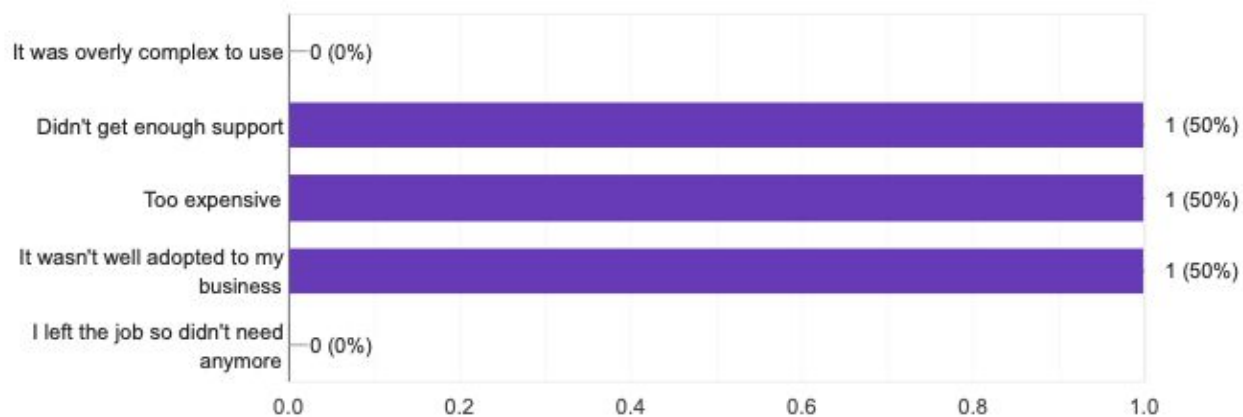


Japonés

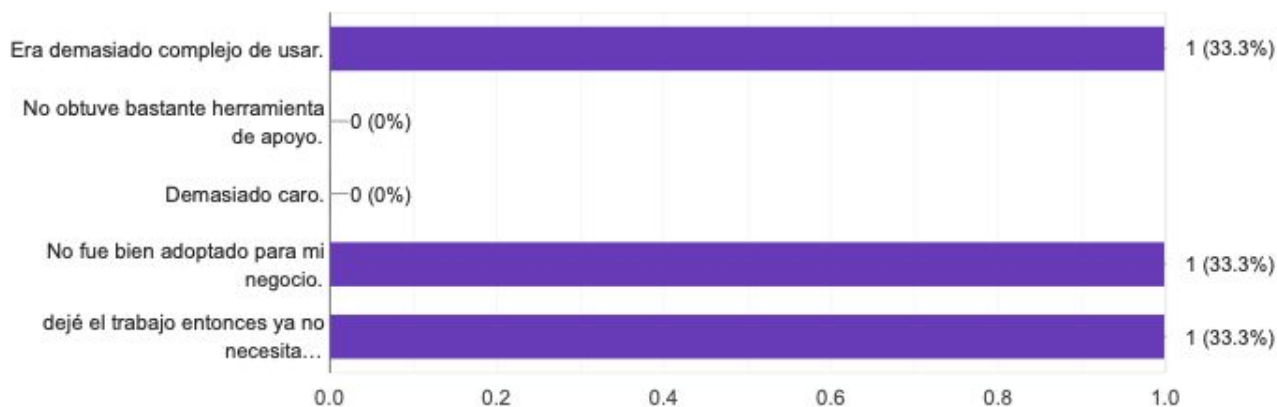


Pregunta 8: Si ha contestado "Sí, he usado alguna vez" en tercera pregunta, ¿Cuál es la razón principal de dejar de usarla?

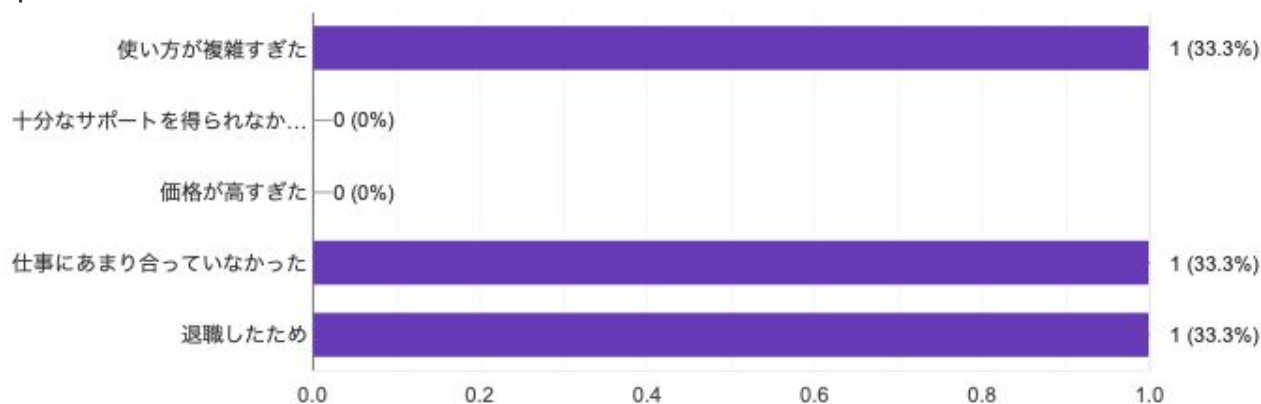
Inglés



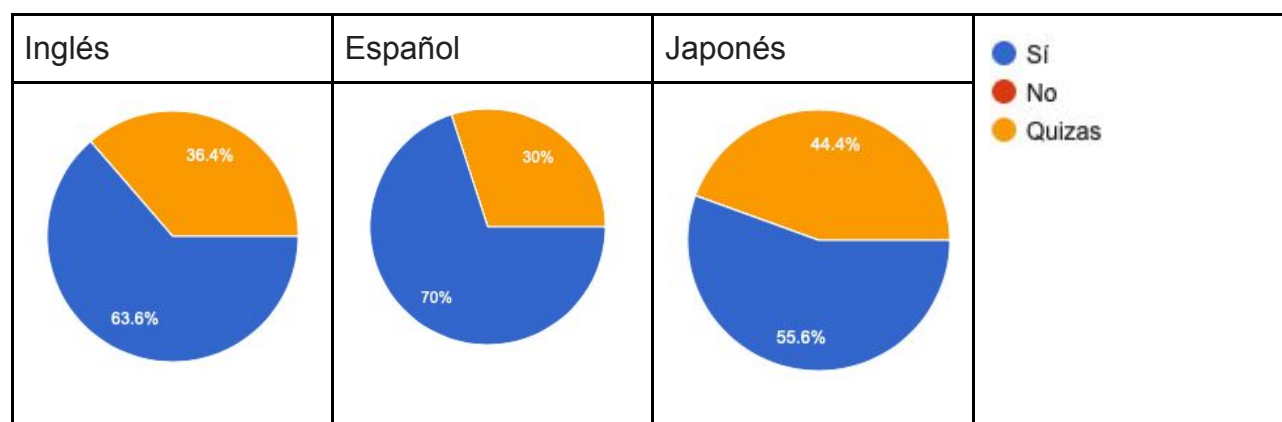
## Español



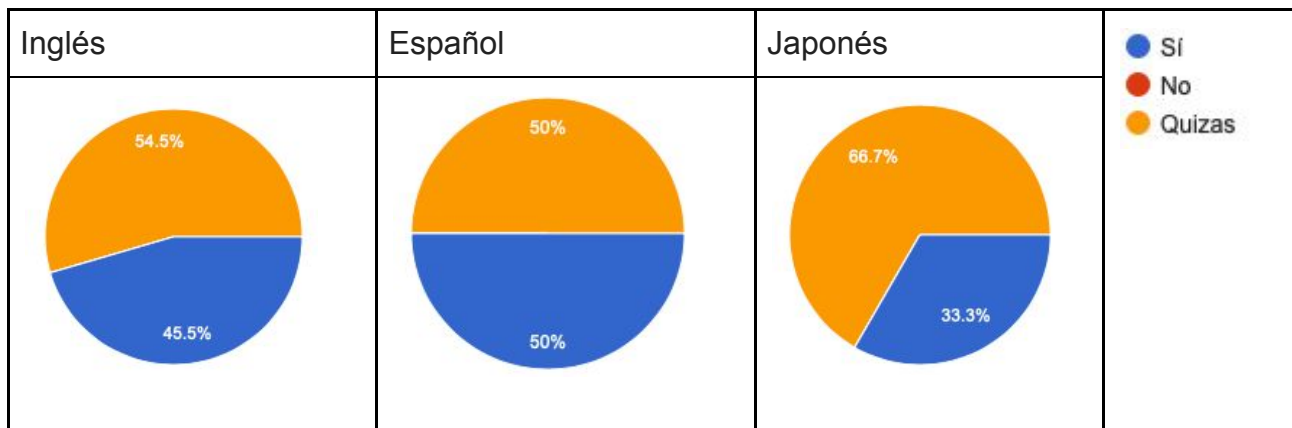
## Japonés



Pregunta 9: ¿Después de leer descripción de mi proyecto, le interesa usar?



Pregunta 10: ¿Después de leer descripción de mi proyecto, recomendaría a alguien?



## 4.6.- PLANIFICACIÓN DEL TIEMPO

Como estaba planeado, era necesario realizar un anteproyecto a fines de diciembre o al menos hasta el final del segundo trimestre. Aquí hay una idea aproximada del plan después de eso.

- Diseñando varios aspectos diferentes, incluyendo interfaz de usuario, usabilidad, producción de contenido y diseño gráfico.
- Redacción de documentación para el proyecto.
- Diseñando una base de datos e ingresando los datos que deben completarse para que las ROOTLETS funcionen, como la tabla de estado y tipo de personalidad.
- Crear un proyecto angular, que es la parte más importante y principal de este proyecto final, obviamente.
- Prueba de funcionalidad del proyecto.
- Haciendo diapositiva para la presentación.

Aquí hice una tabla de horarios aproximada, los seguimientos se refieren al día de la tutoría. Esto es solo una guía, durante el diseño del proceso y la base de datos puede necesitar ser cambiada, la documentación puede ser cambiada debido al cambio del diseño.

| Tarea            | Enero | Marzo | 1° seguimiento | 2° seguimiento | 3° seguimiento | 4° seguimiento | 5° seguimiento | Día de entrega |
|------------------|-------|-------|----------------|----------------|----------------|----------------|----------------|----------------|
| Anteproyecto     |       |       |                |                |                |                |                |                |
| Diseño           |       |       |                |                |                |                |                |                |
| Documentación    |       |       |                |                |                |                |                |                |
| Base de datos    |       |       |                |                |                |                |                |                |
| Proyecto Angular |       |       |                |                |                |                |                |                |
| Test             |       |       |                |                |                |                |                |                |
| Presentación     |       |       |                |                |                |                |                |                |

## 4.7.- ESTIMACIÓN DE COSTES

Para mi proyecto, requiere dominio y licencia de base de datos. En primer lugar, el dominio tiene una gran variedad de precios desde 13 euros hasta 140 euros. Por ejemplo, ".es" cuesta 14,95 euros al año y ".com" cuesta 13,95 euros al año. Creo que es más apropiado usar ".com" para mi proyecto, ya que es conocido en todo el mundo y no es necesario limitar el objetivo de marketing solo a la empresa española. Además, cuando se trata de la licencia de la base de datos, podemos elegir entre diferentes compañías, utilizando un servidor físico o un servidor en la nube, y el tamaño del almacenamiento. Aquí elegiría MySQL Enterprise Edition, que cuesta 5000 USD por año, porque hasta ahora solo tengo conocimiento de MySQL database. Además, la Enterprise Edition tiene más estabilidad y soporte ya que es más adecuado para gestionar datos personales. En el futuro, podría decidir cambiar a la base de datos con un servidor en la nube. Investigué Azure que es una base de datos SQL y también se recomendó en la taller de la sesión de nube organizado por Accenture para el comienzo de la computación en la nube. Cuesta 1.173,12 por año. Para programador que están empezando su trayectoria profesional y tienen menos de 1 año de experiencia, el salario medio es de 17,000-21,000 euros brutos anuales que 8.14-12.15 euros. Para la construcción de esta aplicación web, tengo tres meses completos y puedo decir que pasaría entre 200 y 250 horas, probablemente más. En consecuencia, mi salario estimado sería de 1628-2035 euros, que multiplicaron las horas de trabajo por 8.14.

## 4.8.- ESTIMACIÓN DE INGRESOS

Según la encuesta en español, el 70% de los encuestados están interesados en usar ROOTLETS. Este número 70% en realidad incluye 20% estudiantes y 40% empleados que no están a cargo de tomar la decisión dentro de la empresa, por lo que solo el 40% de este 70% estas personas realmente comprarán mi aplicación. Si toda la empresa existe en España se prueba y el mismo porcentaje de ellos decide felizmente usarla con el plan mensual más barato después del período de prueba, el ingreso será  $(34004 * 0.7 * 0.4) * \text{precio del plan mensual}$  que resulta en 190,422.4 euros por mes. Sin embargo, hay un 40% de ellos que actualmente usan algún software CRM, lo que significa que hay menos posibilidades de comprar uno nuevo, por lo tanto, los sacaré del cálculo. La estimación final de ingreso es de 114.253,44 euros al mes.

En España, aproximadamente el 22% de la población habla inglés, por lo tanto, para obtener un número más preciso, combinaría el resultado de la encuesta en inglés que afectará solo el 22% a la estimación de ingreso. Al final, El resultado es 111.146.862 euros al mes.

## 4.9.- CONCLUSIONES

Al comparar la estimación de gastos e ingresos, descubro que este proyecto puede ser increíblemente rentable. Este resultado me hace reconsiderar el precio de ROOTLET, aunque la estimación de ingreso se calculó con un plan mensual de 20 euros, que era inferior al precio promedio del software CRM en el mercado actualmente. Además, para el software CRM, la viabilidad es esencial. No es muy realista vender ROOTLETS inmediatamente después de la entrega en junio. Sin lugar a dudas, tendré que investigar sobre ciberseguridad, protección de datos y mucho más. Así que esa hora de trabajo para este proyecto será mucho más larga.

Para mi curioso, hice una encuesta en japonés y descubrí que el software CRM es menos conocido o tiene menos importancia en el negocio para ellos. En otras palabras, hay menos competencia y podemos esperar más crecimiento en el mercado en el futuro.

## 5.- Arquitectura de la Aplicación

Quería centrarme en la simplicidad. Dado que el sistema CRM siempre es complicado de usar y muchos usuarios se sienten asustados o pereza para implementar, yo estaba más enfocado en el diseño simple.

### 5.1.- RECURSOS Y TECNOLOGÍAS EMPLEADAS

Elegí Angular porque es una de las tecnologías más importante que se considera que tenga mayor futuro. El uso de bases de datos lo realicé con MySQL lo cual hemos estado usando mas en el curso y me siento más cómoda para usar. En front, utilicé html y scss. Dentro de pagina tonto como las funcionalidades, los servicios y los modelos, y back utilicé typescript.

Además, el parte de back, Servidor, lo realice con node y express web framework, mysql-promise y REST api para la gestión de bases de datos.

Para la generación de pdf utilice librería de pdfmake.

Enviar correo electrónico lo realice con nodemailer.

### 5.2.- DISEÑO INICIAL

Hay cuatro roles: administrador, desarrollador de negocios, desarrollador / diseñador y cliente.

El administrador puede gestionar todos los contactos del cliente, incluidos clientes y empleados, es decir, crear nuevos contactos, modificar y eliminar los contactos y ver los detalles de contacto.

El desarrollador de negocios puede agregar y modificar contactos, acceder a los detalles del proyecto, sin embargo, no puede borrar sus contactos. Además, puede ver los

proyectos y facturar los proyectos, aplicar promociones al proyecto, crear y modificar promociones.

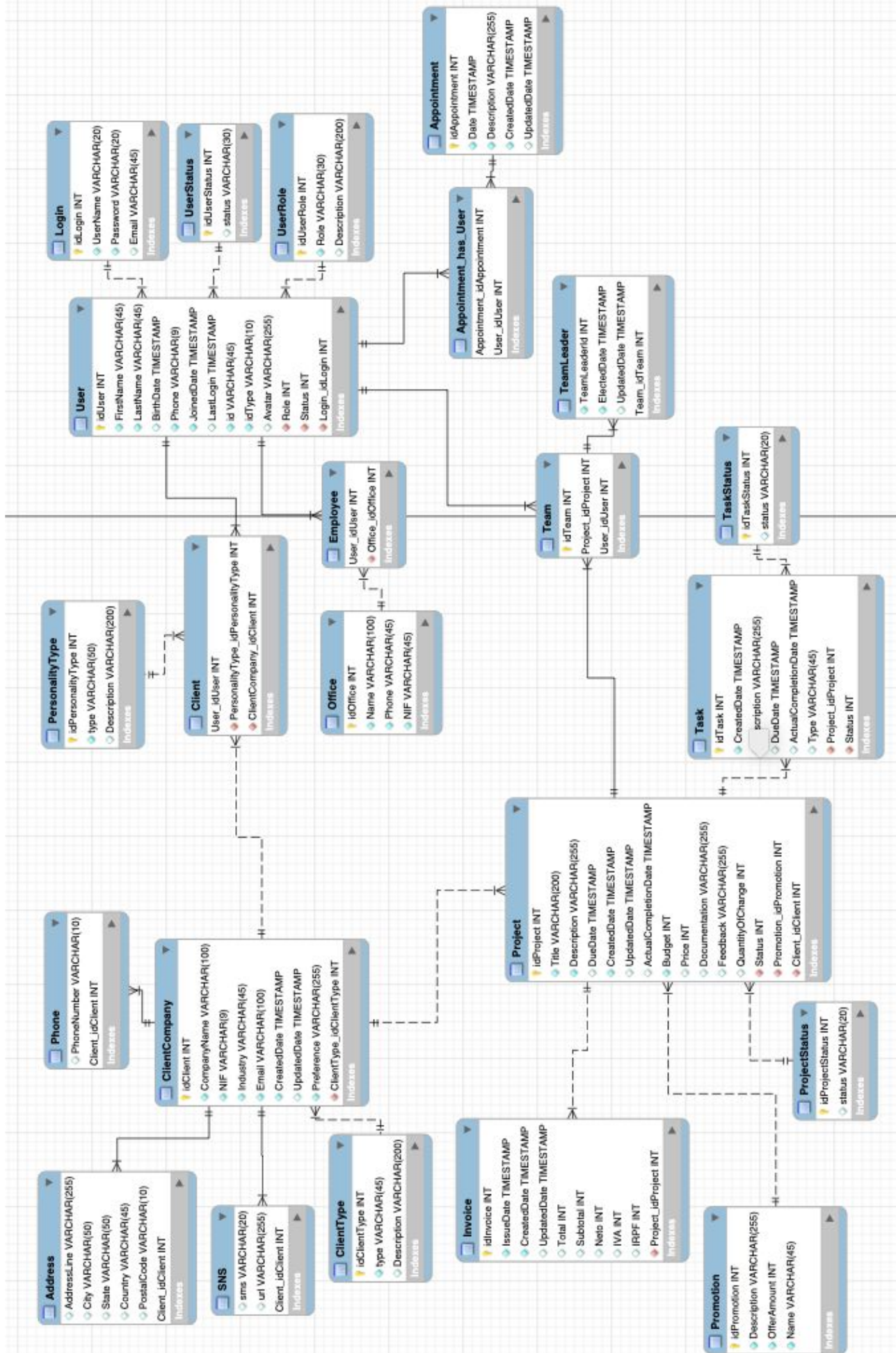
El desarrollador y el diseñador pueden ver los detalles de su proyecto y agregar, modificar, eliminar y gestión de tareas.

El cliente solo puede acceder a los detalles del proyecto y seguir su progreso. no se le permite acceder a todos los detalles de las tareas sino a las tareas principales.



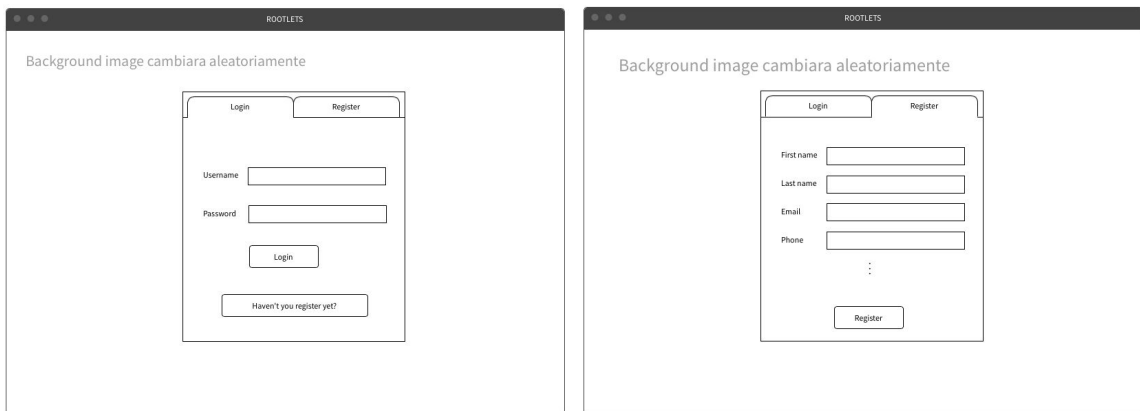
[illegible]

- Diagrama E-R

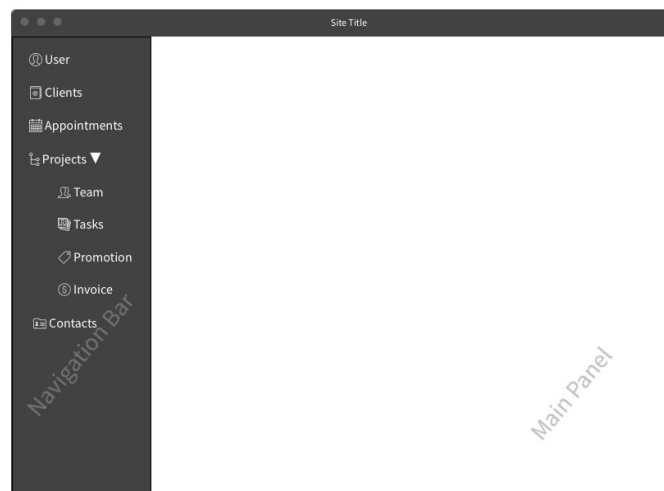


- Esquema de las pantallas que tendrá tu interfaz gráfica.

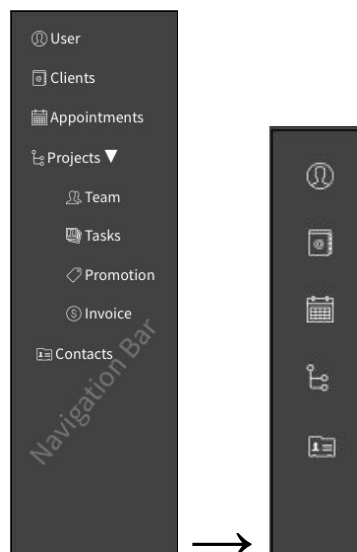
Estos Login y Registro son primeras páginas que aparezcan cuando usuario entra a ROOTLETS. El imagen del fondo cambiará aleatoriamente.



Una vez inicie sesión, entra página principal



Barra de navegación puede ser escondida



Página de User

User

Clients

Appointments

Projects

Team

Tasks

Promotion

Invoice

Contacts

Site Title

User Information

Log out

Change Avatar

John Smith

Date of birth : DD/MM/YYYY

Email : john@example

Username : john

Password : .....

Phone number : 000000000

Joined date : DD/MM/YYYY

ID : DNI 11111111A

...

Edit

Add office

Página de Clientes listados

User

Clients

Appointments

Projects

Team

Tasks

Promotion

Invoice

Contacts

Site Title

New Regular Potential

Add New Client

Company name

Info Appointment Current project Contact

Company name

Info Appointment Current project Contact

Company name

Info Appointment Current project Contact

Página de Cliente

User

Clients

Appointments

Projects

Team

Tasks

Promotion

Invoice

Contacts

Site Title

Client company name Regular client

NIF : 00000000A

Industry : Retail

Email : example1@example.com

example2@example.com

Created date : DD/MM/YYYY

Updated date : DD/MM/YYYY

Preference : Simple, Clean

...

See order history

Check current project

Edit

Appointment

Contact

Página de Appointments

User

Clients

Appointments

Projects

Team

Tasks

Promotion

Invoice

Contacts

Site Title

Today Upcoming Done

Add New

DD/MM/YYYY HH:MM with Company name

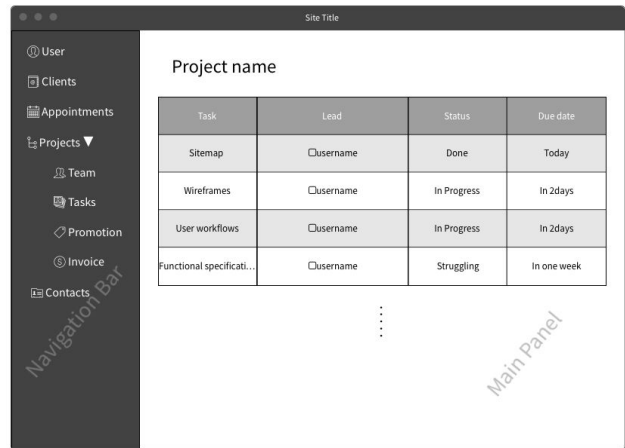
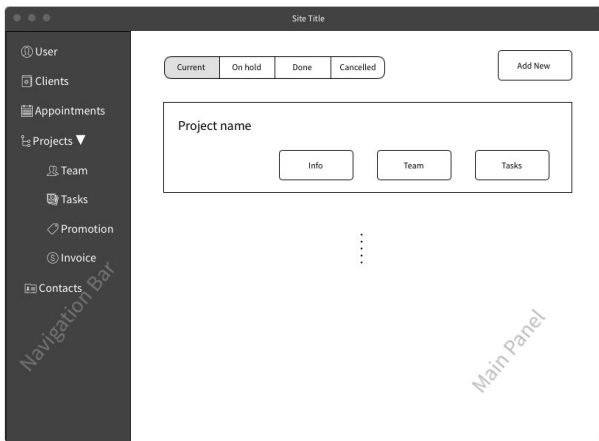
Info

Contact

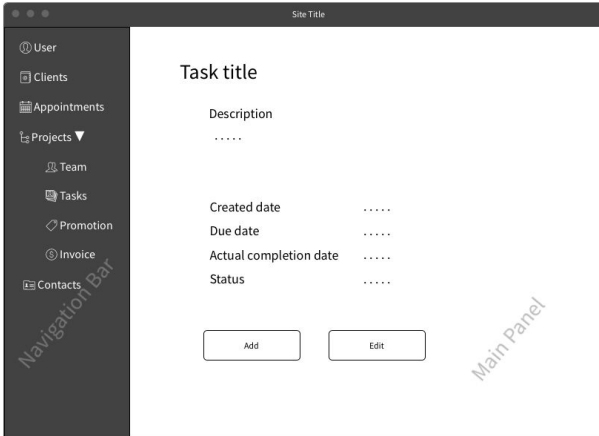
...

Página de Projects listado

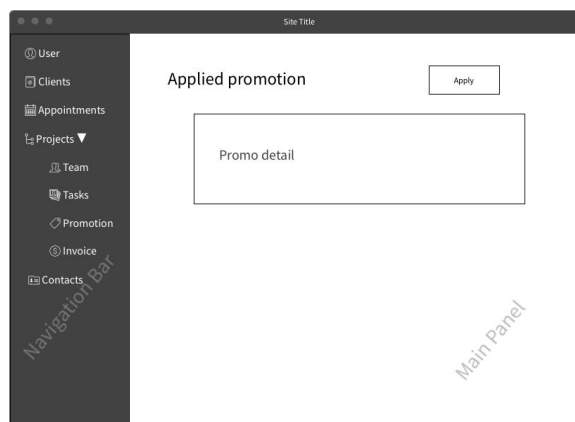
Página de Tasks de proyecto



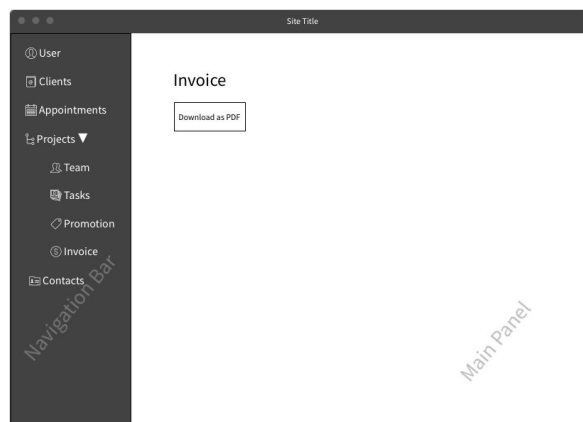
Página de Detalle de Task



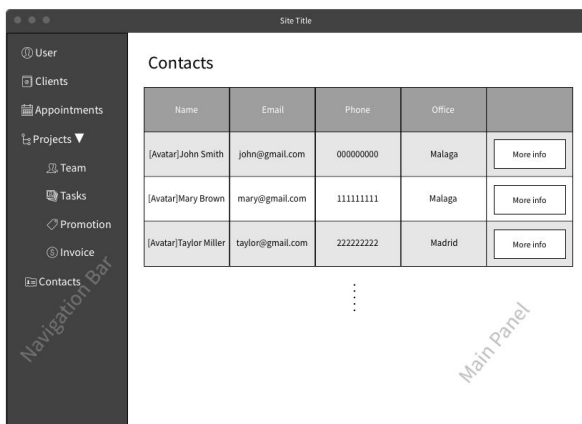
Página de Promotion



Pagina para imprimir invoice



Página de contactos



Página de Office al que solo puede acceder el administrador

Office

|        |                      |
|--------|----------------------|
| Name:  | <input type="text"/> |
| Phone: | <input type="text"/> |
| NIF:   | <input type="text"/> |

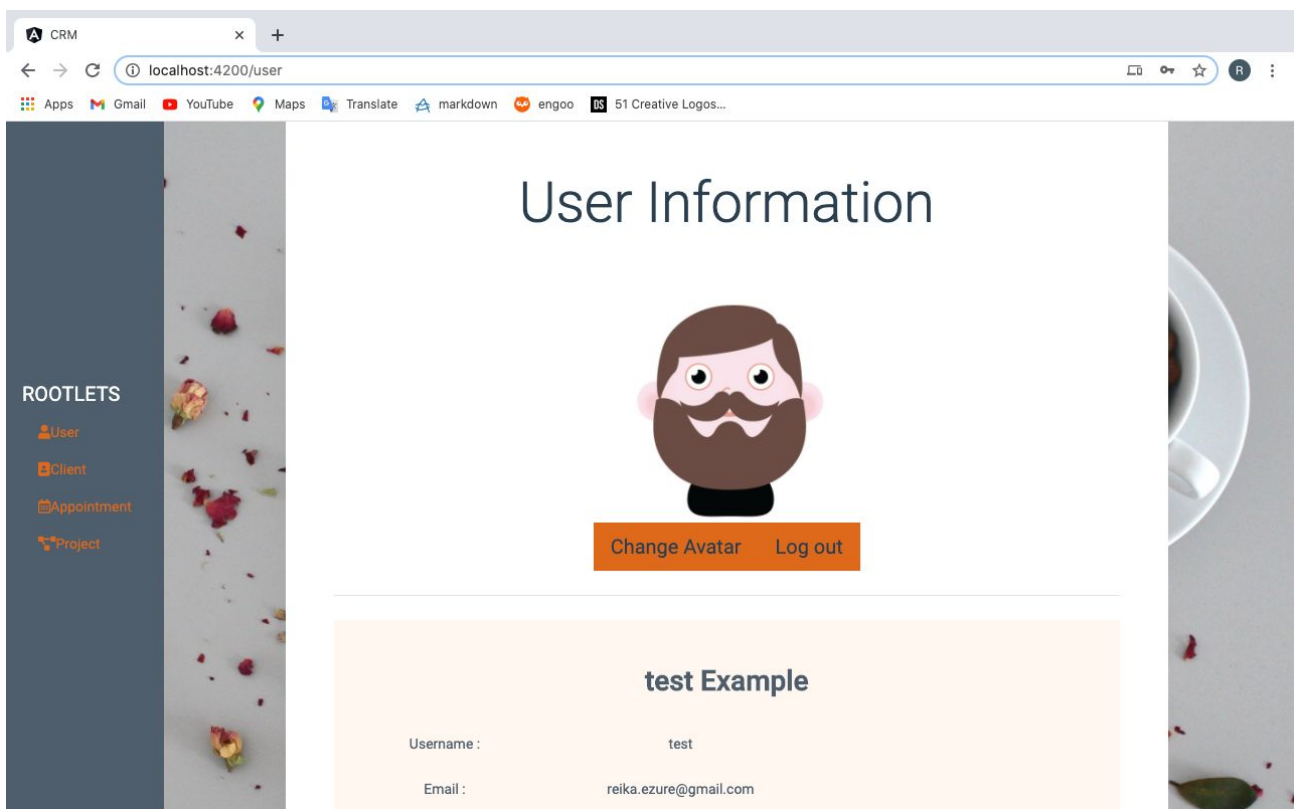
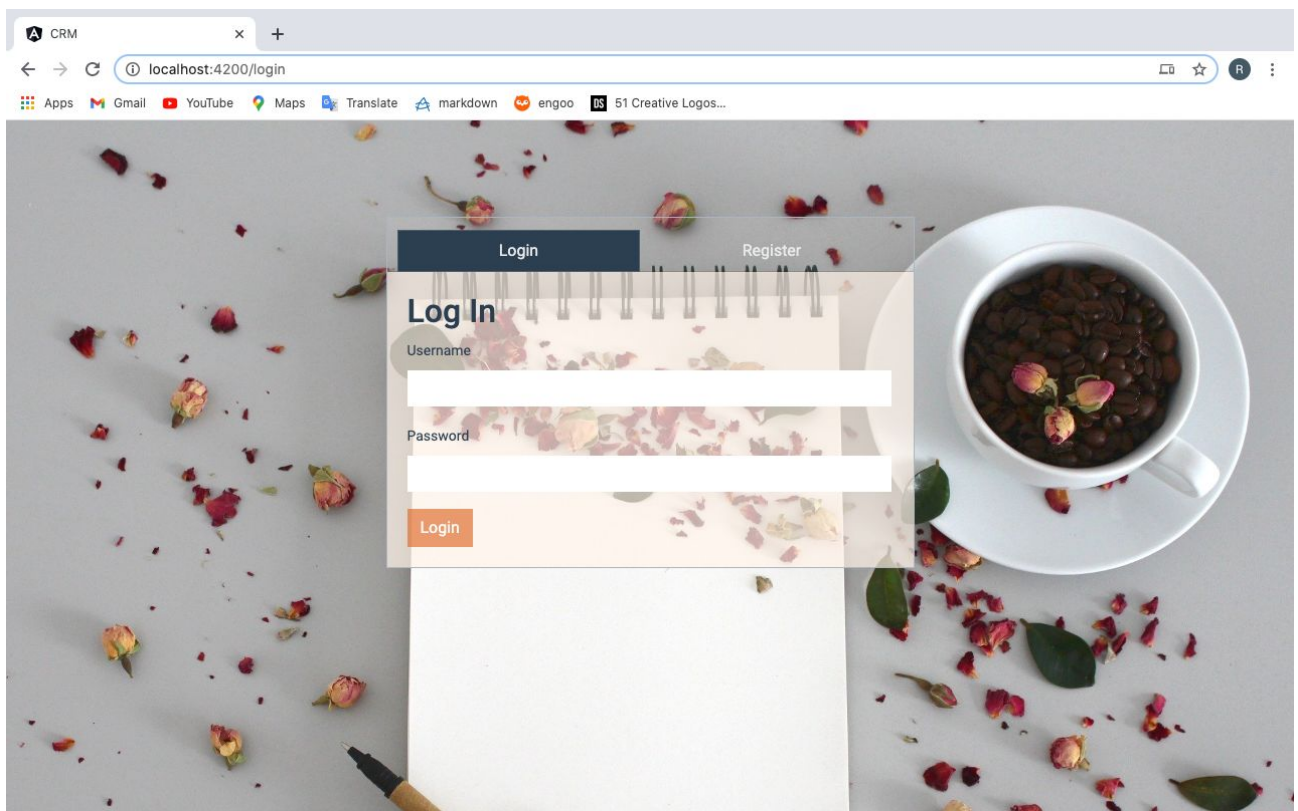
Main Panel

### 5.3.- DISEÑO FINAL

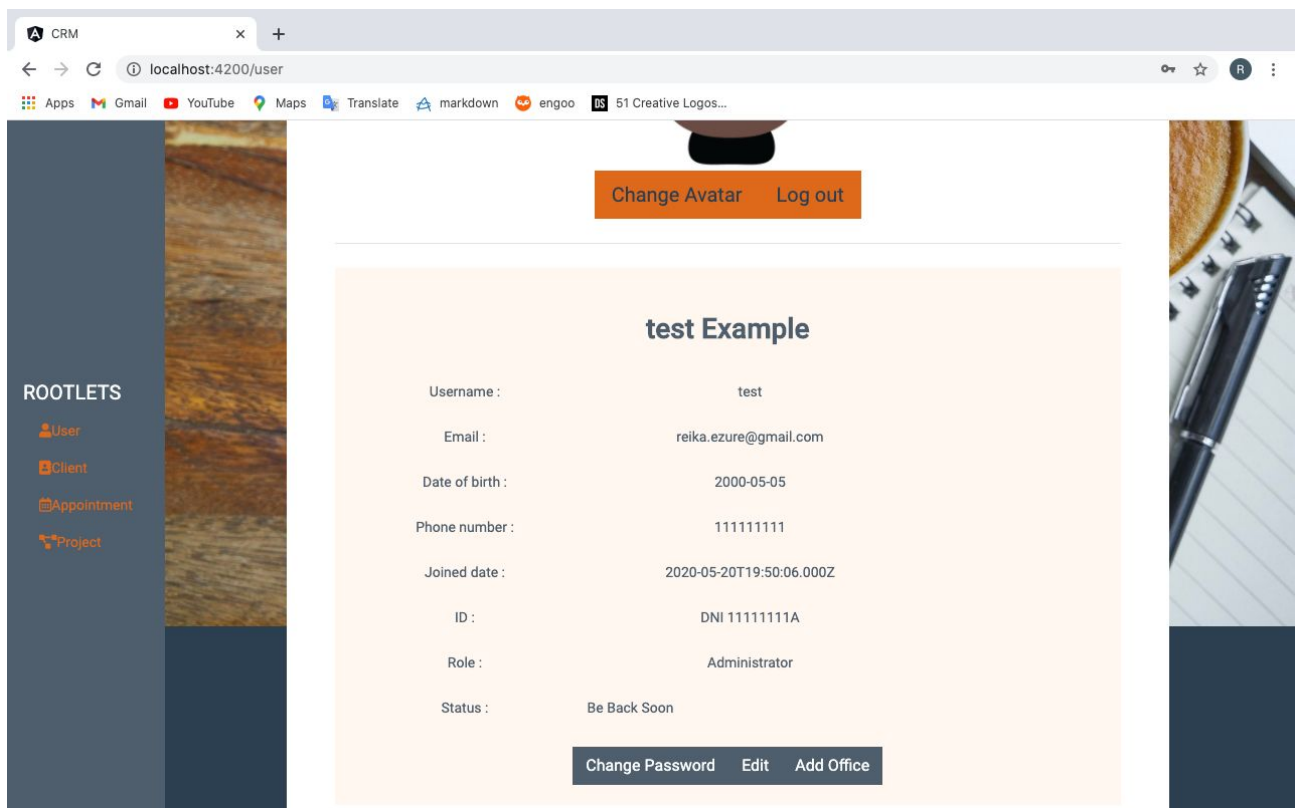
Al principio, planteaba hacer la funcionalidad que cierta role permite ciertas funcionalidades a otro rol, por ejemplo, el desarrollador de negocio puede agregar y modificar los datos de contacto siempre cuando tenga permiso de administrador. Sin embargo, el proceso de pedir permiso, esperar hasta otro role le da permiso, recibe permiso y realizar gestión son demasiado largas y complica la implementación al usuario. Como mi objetivo principal es crear CRM más simple, intuitivo y fácil para usar, esta funcionalidad actuaría contradictoriamente hacia mi objetivo aunque hubiera sido interesante para hacerla.

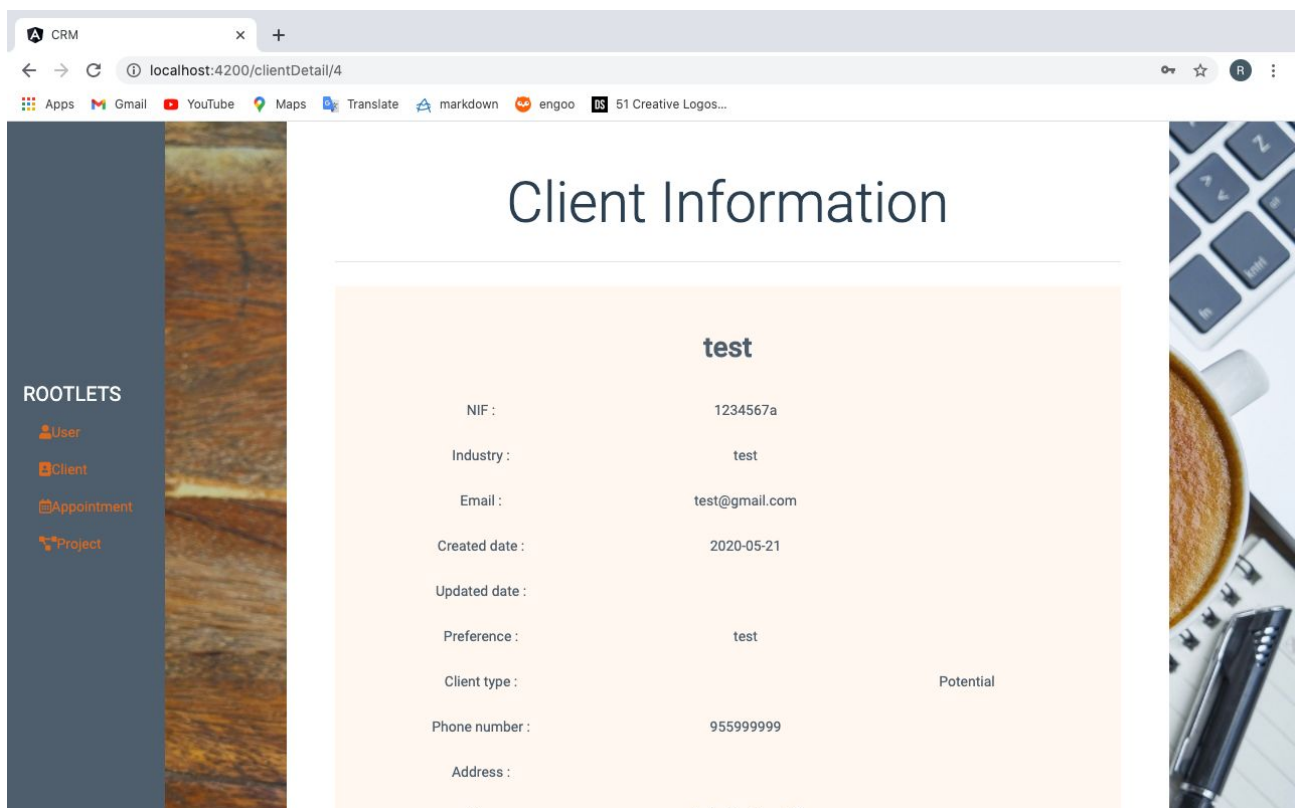
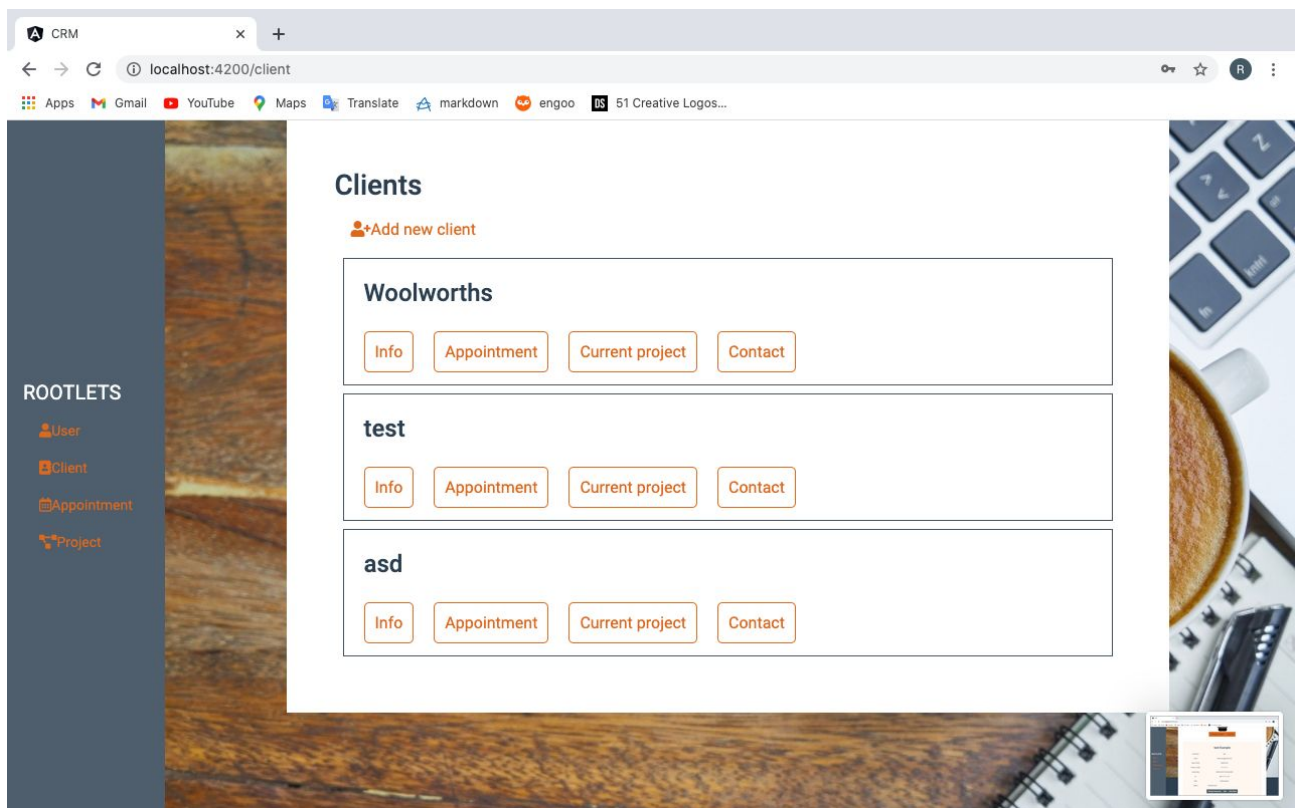
El diseño de interface no cambié mucho. Manteniendo el diseño inicial como donde coloca los botones, menú y texto. Diseñe la interfaz más simple y menos texto posible como la gente no lee pero escanea la página y se agobia con muchos textos. Cada vez entra a login cambia el imagen de fondo. Añadí la funcionalidad de avatar personalizada para darle al usuario la gracia. Voy a dejar las capturas de pantalla.

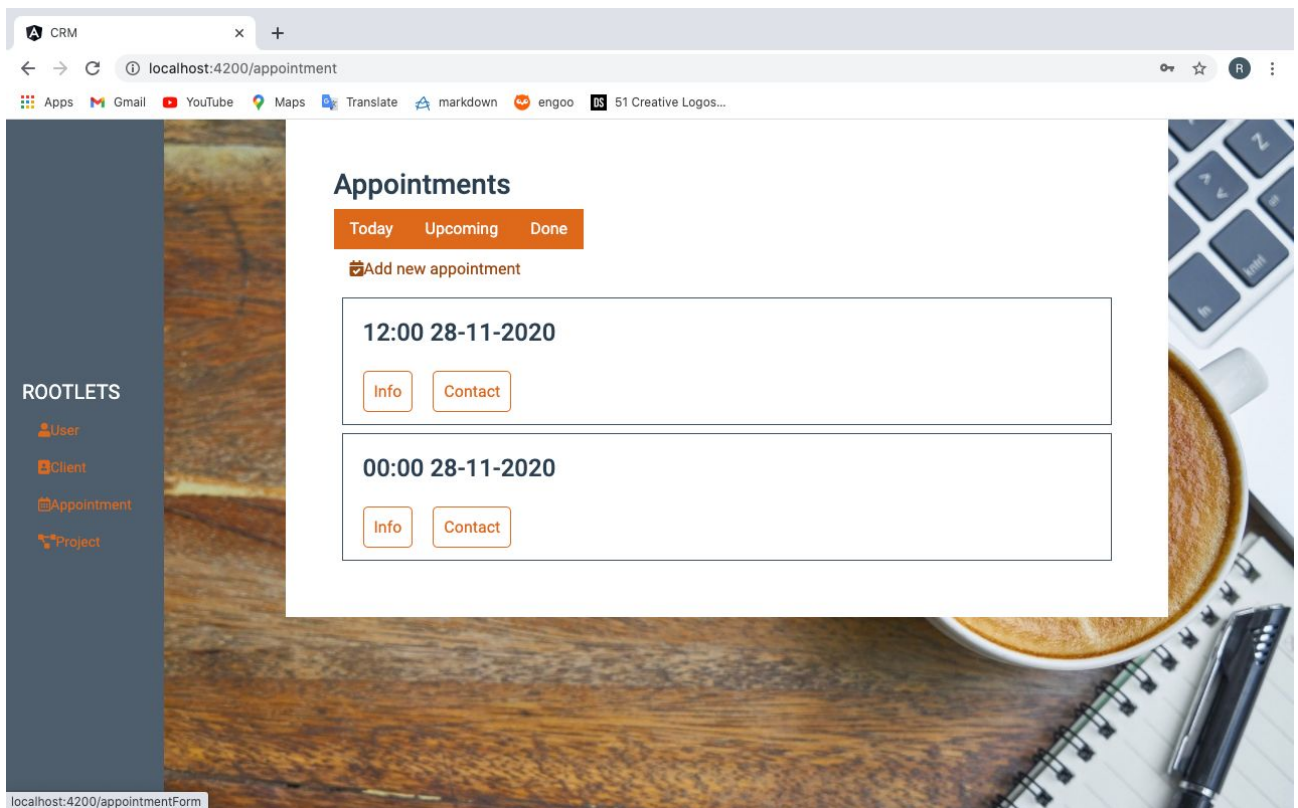
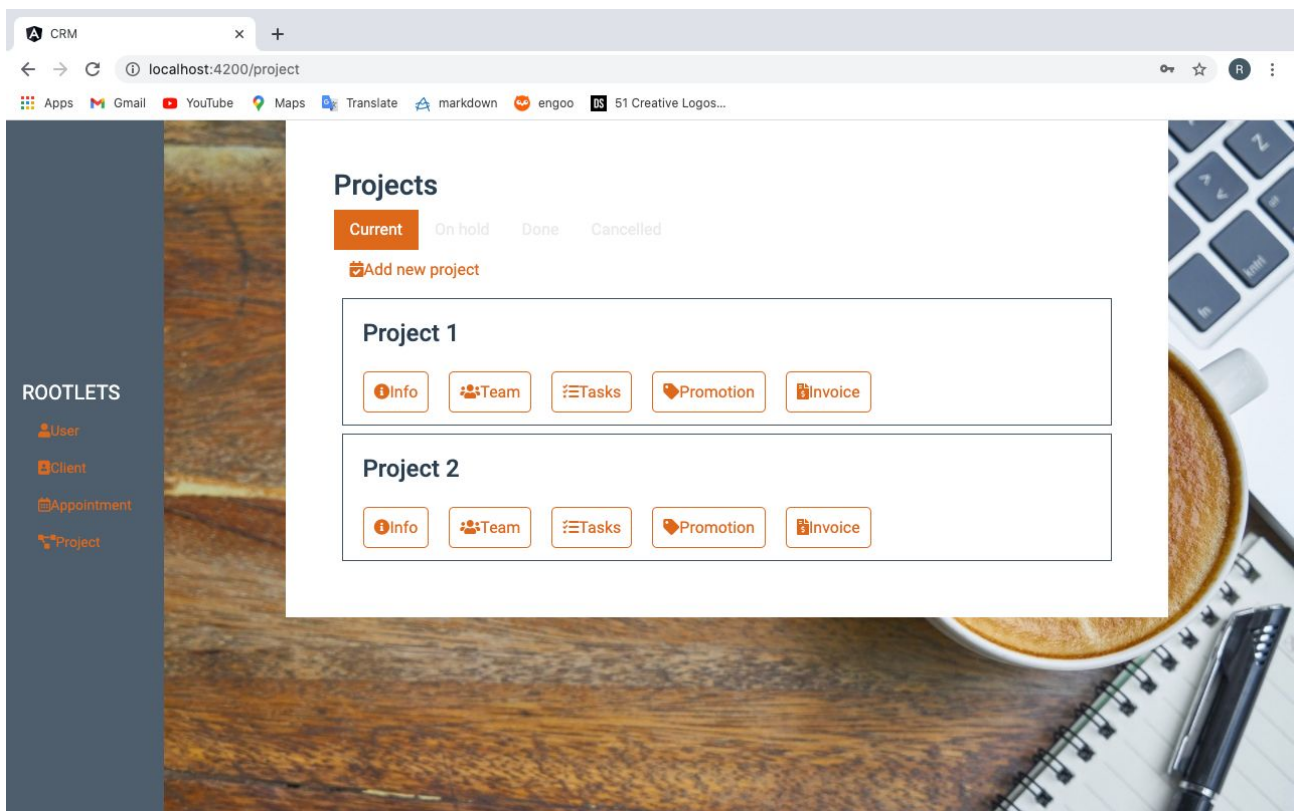


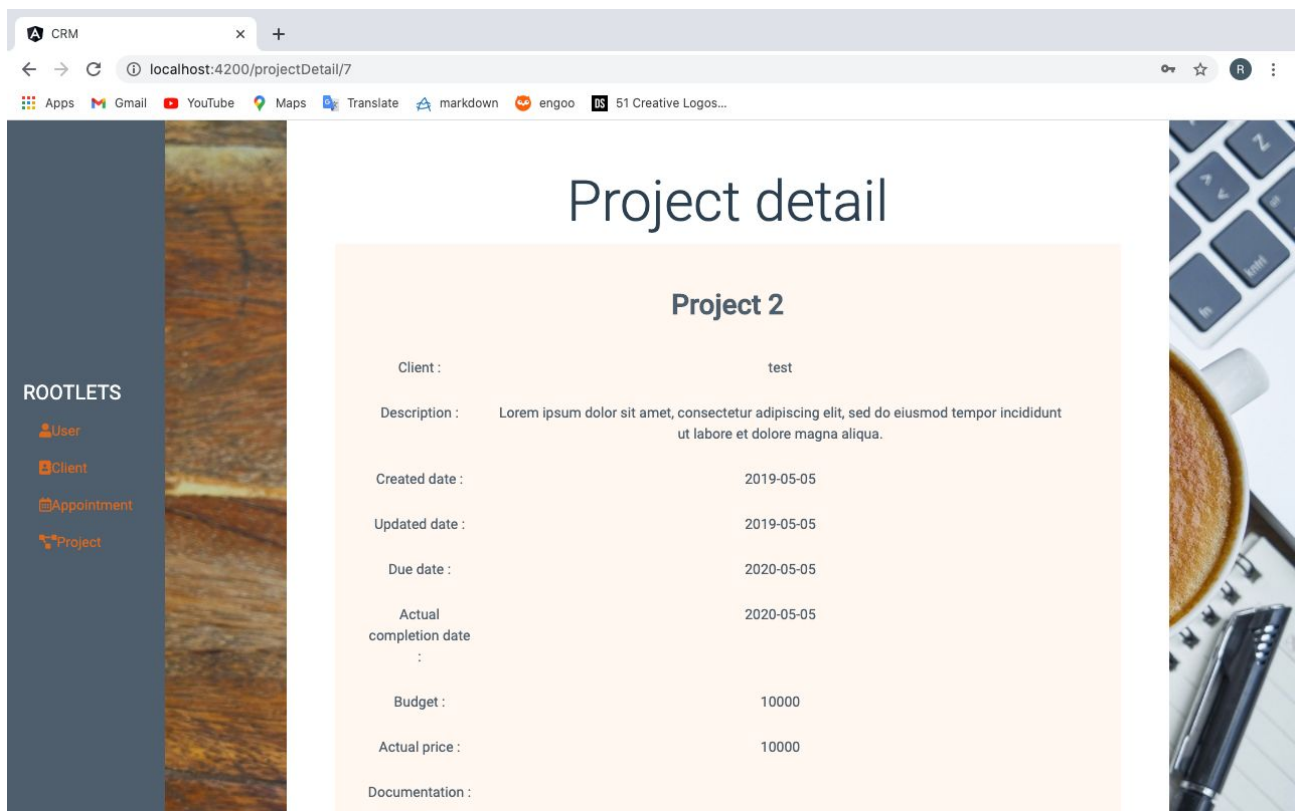




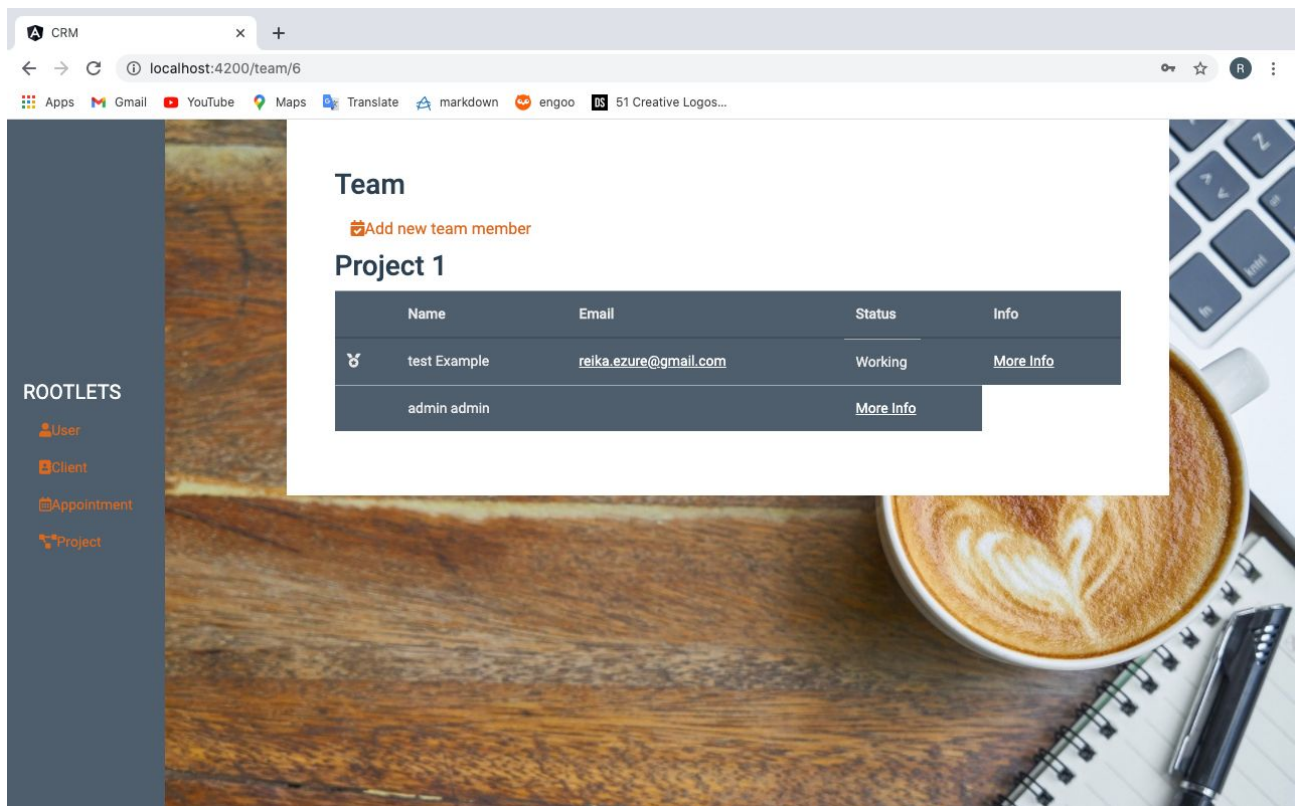
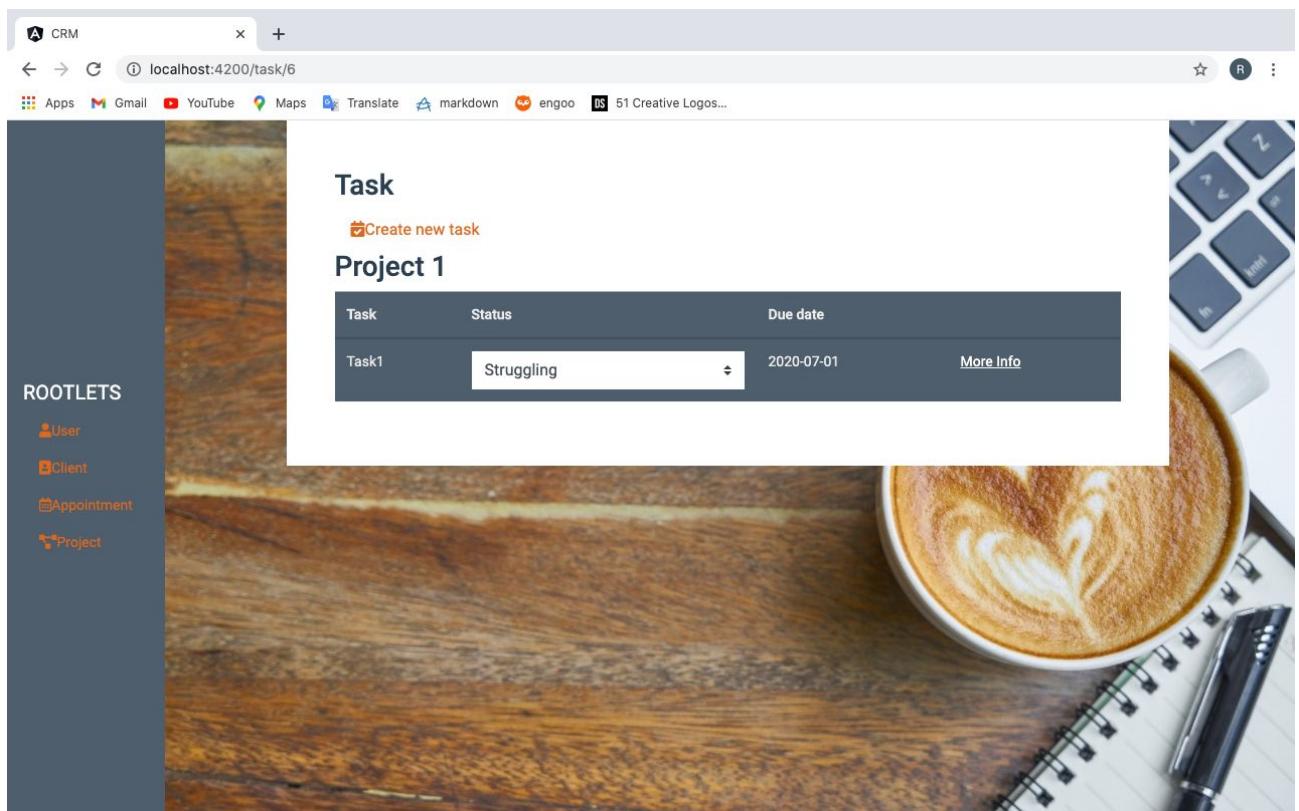


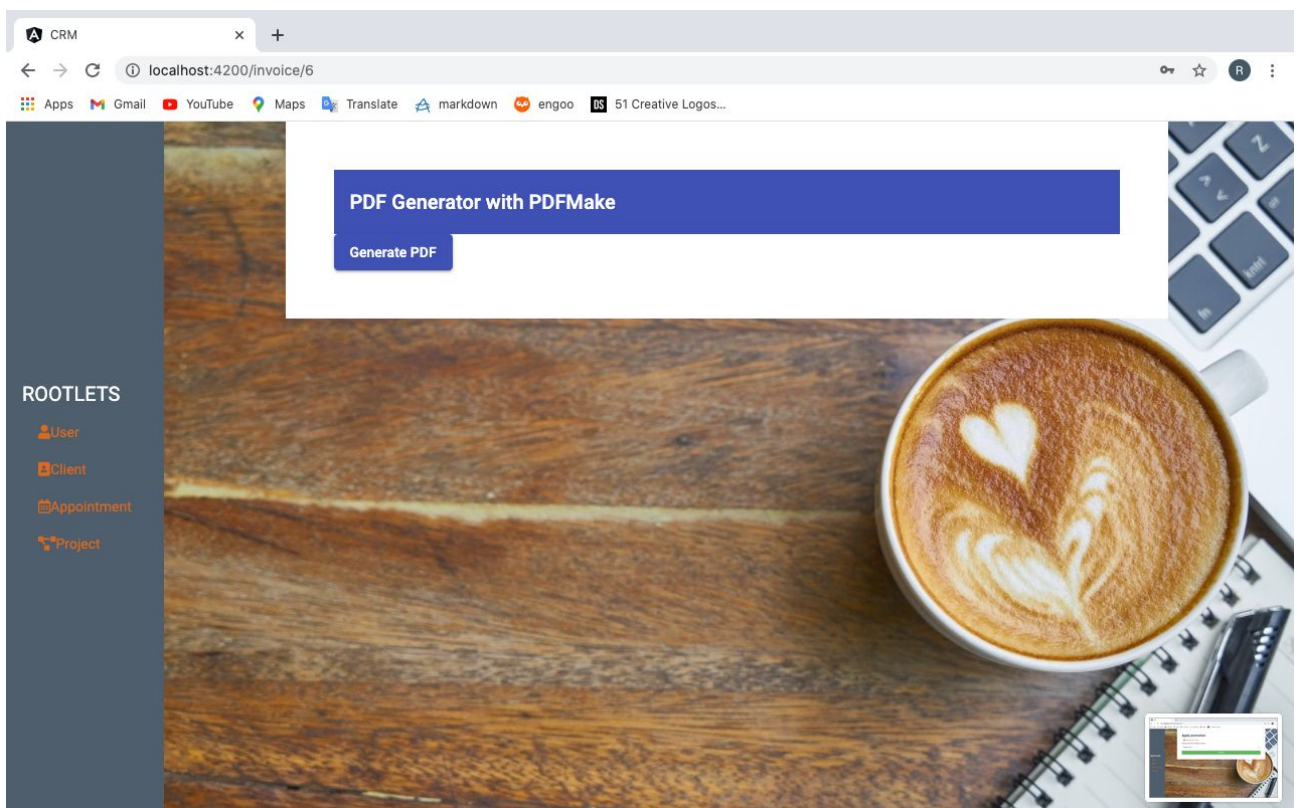
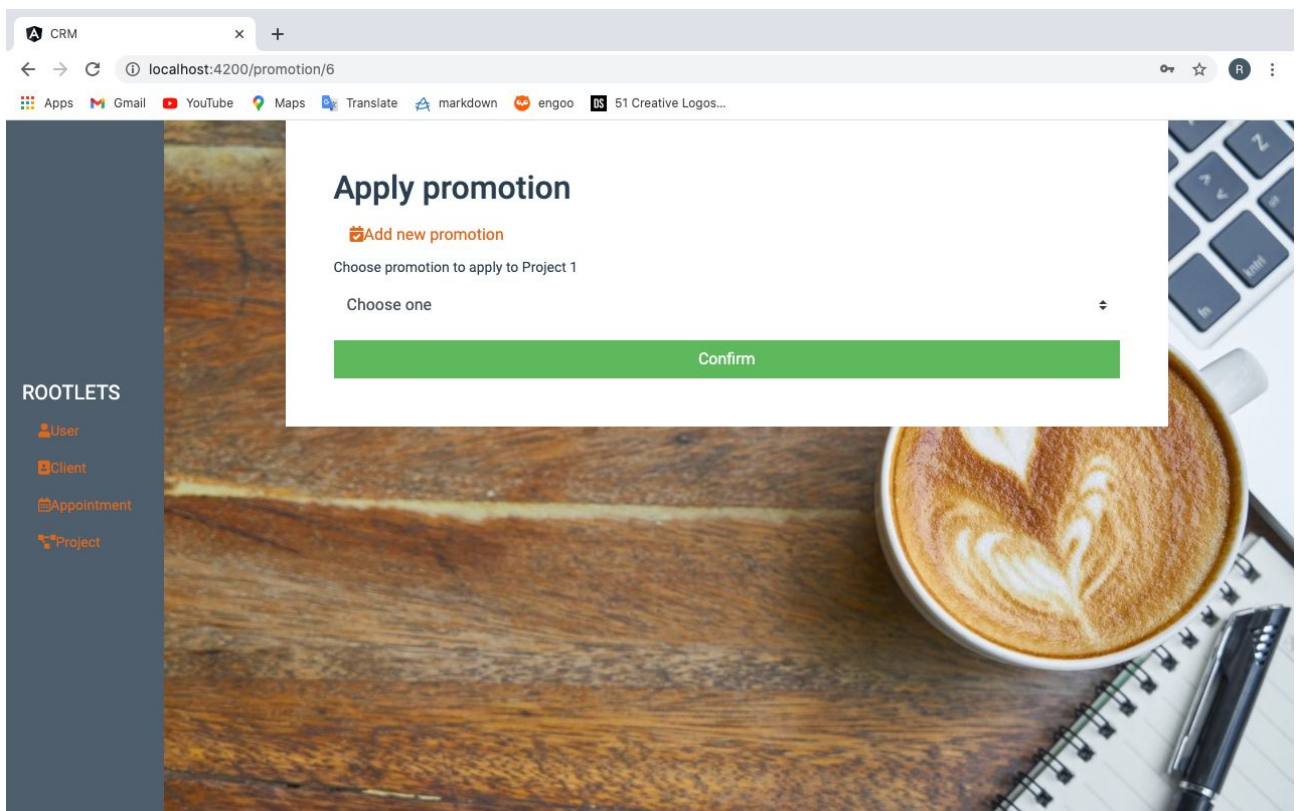












## 6.- Futuras Mejoras y Conclusiones

Como nunca estoy realmente satisfecha al 100% con mi trabajo y siempre pienso que debe haber algo que pueda mejorar y no existe ningún trabajo perfecto y completo, voy a indicar varias cosas que me hubiera gustado implementar:

Guardar contraseña cifrada en base de datos y cookie.

Por razones de seguridad, creé una tabla diferente llamada 'Login' en la base de datos que almacena datos de usuario, solamente nombre de usuario, contraseña y correo electrónico. Entonces, en caso de que alguien pueda hackear e entrar a la tabla 'Login', hay una tabla más para hackear si quieren tener información importante real de cada usuario.

Diferente método de acceso.

Podría enviar 4 o 6 dígitos al número de teléfono y el usuario debe introducir esos dígitos para iniciar sesión, en lugar de introducir el nombre de usuario y la contraseña.

Autenticación de doble factor

No estoy seguro de si este tipo de sistema también necesita la autenticación de doble factor por ley, pero sería interesante si pudiera haber aprendido a hacerlo.

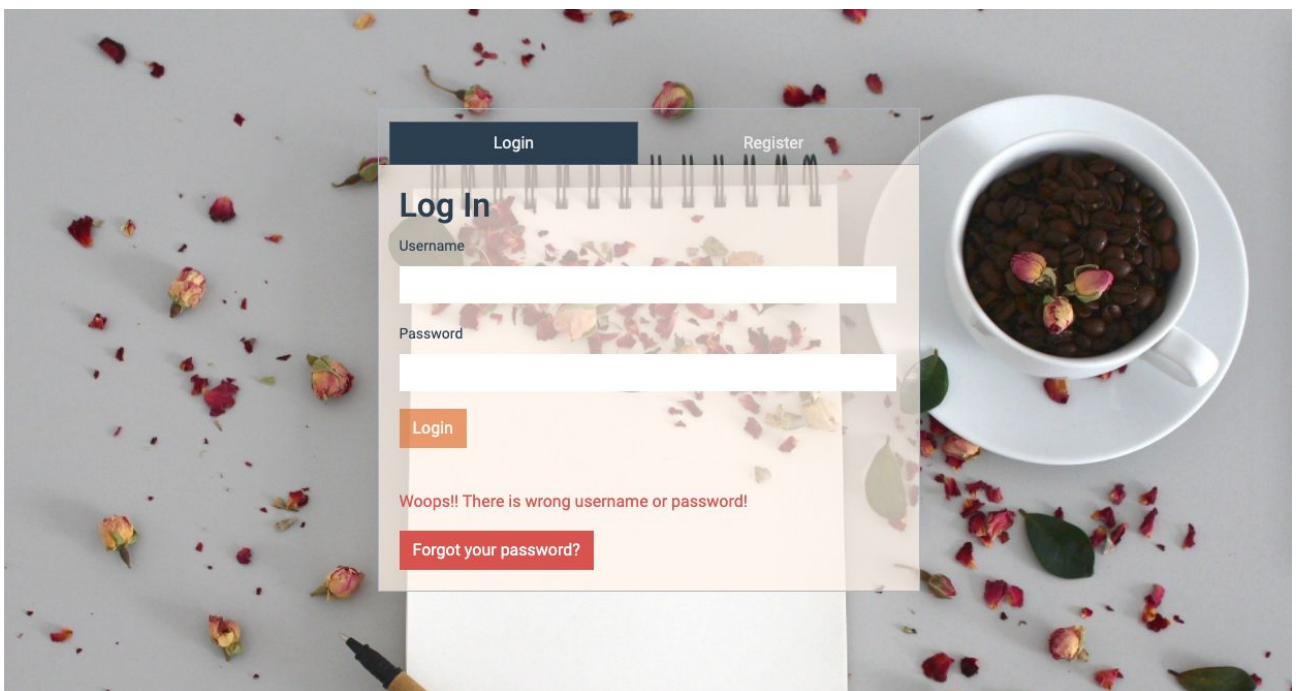
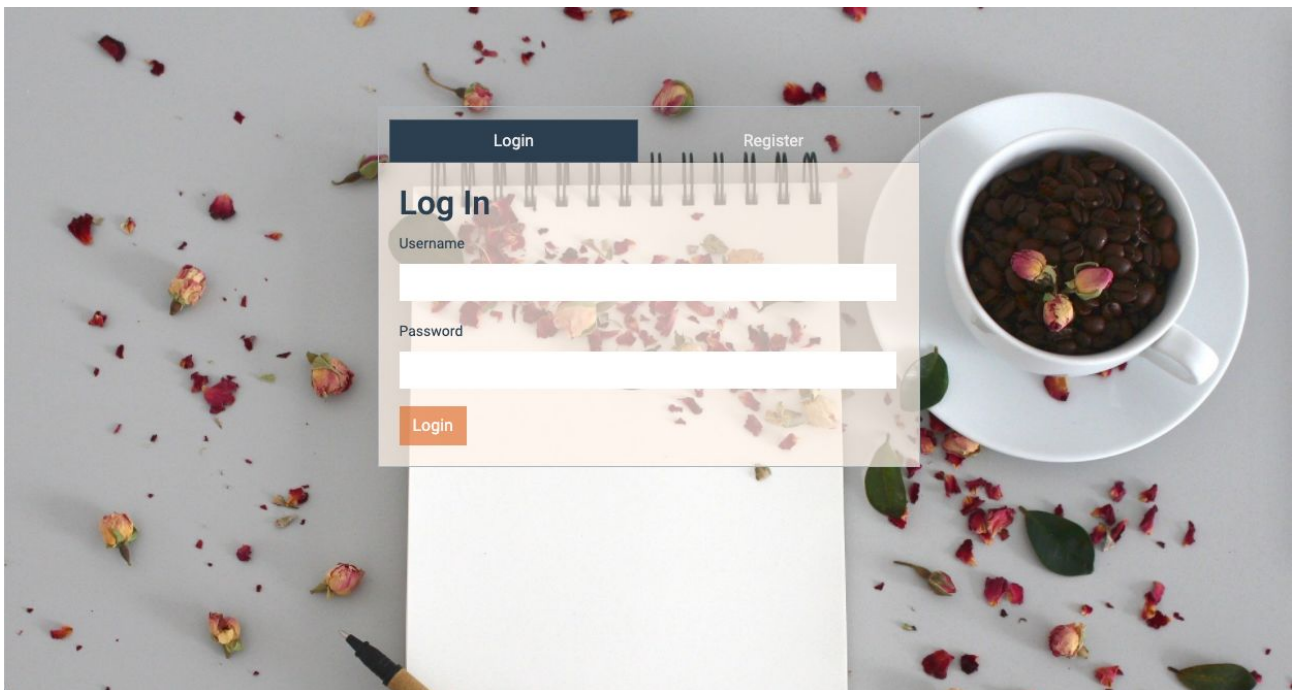
## 7.- Pruebas y Resultados

Las prueba que realice son las siguientes:



## Login

Al iniciar sesión si el usuario se equivoca el nombre de usuario o la contraseña, sale el mensaje que le informa están equivocados y botón para recuperar la contraseña perdida.



El nombre de usuario y la contraseña se guardarán en la cookie durante 1 día. Entonces, si el usuario cierra la página sin cerrar la sesión y vuelve, no se le pedirá al usuario que ingrese su nombre de usuario y contraseña nuevamente. Cuando cierra la sesión, cookie se borrarán.





Al registrar el usuario debe ingresar la contraseña dos veces para confirmar. Si esas contraseñas no coinciden, el mensaje aparecerá directamente sin necesidad de enviar formulario.

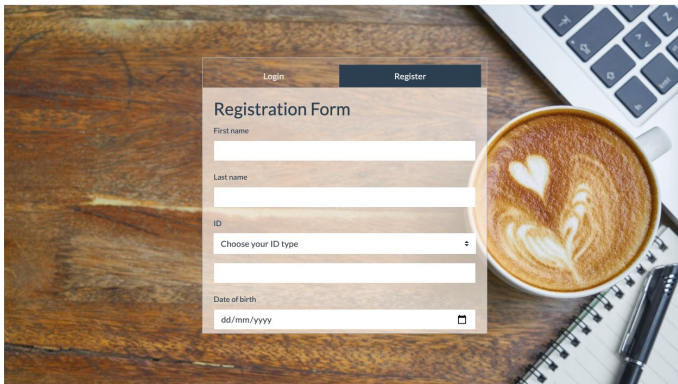


The image shows a registration form with two password fields. The first field is labeled 'Password' and the second is labeled 'Confirm Password'. Both fields contain masked text (dots). A red border highlights the 'Confirm Password' field, and a red exclamation mark icon is visible in the bottom right corner of this field. Below the fields, a red error message states: 'Passwords do not match'.

## 8.- Manual Usuario

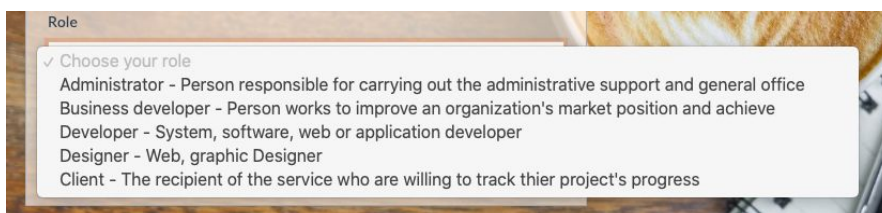
### Configuración para empleados

Para comenzar desde cero, deberá registrarse como cuenta de administrador para poder acceder a la función que sea necesaria. Para registrarse, haga clic en la pestaña de registro e inserte sus datos.



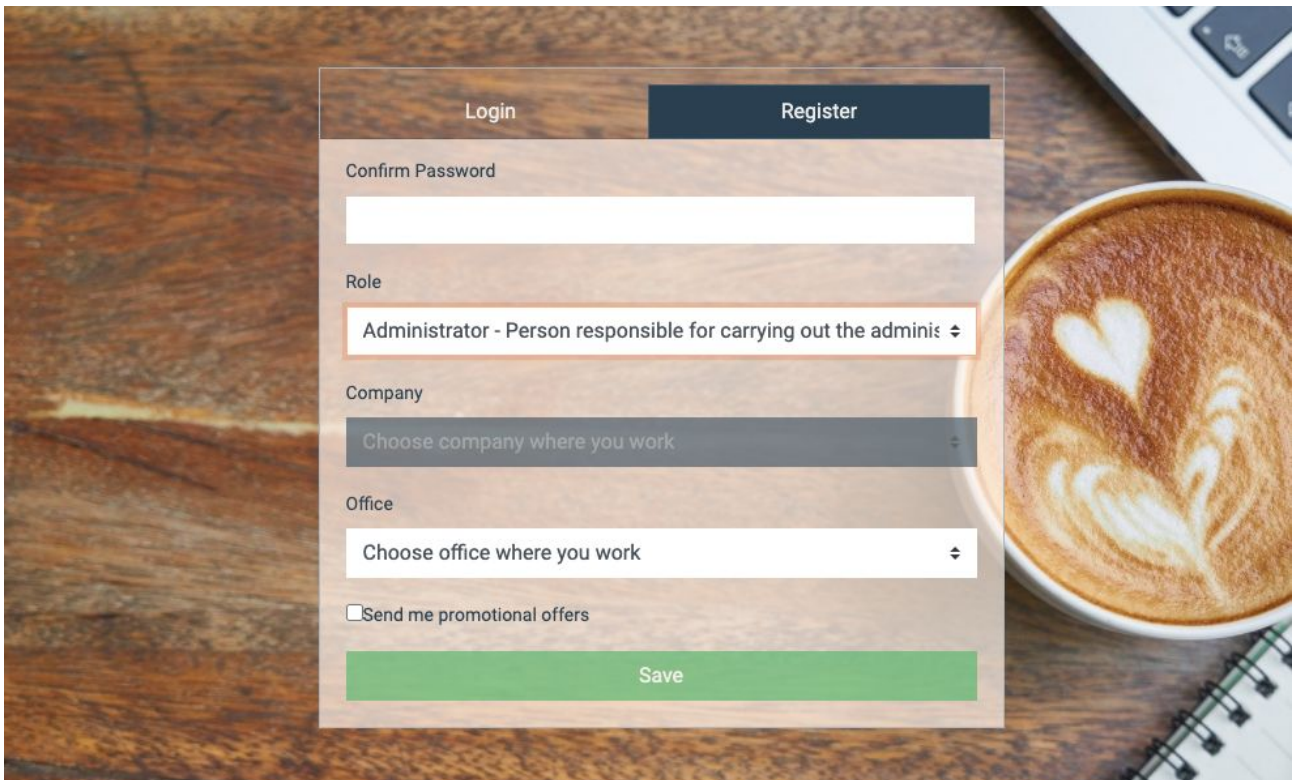
The image shows a registration form overlay on a background of a laptop and a cup of coffee. The form has two tabs: 'Login' and 'Register'. The 'Register' tab is active. The form is titled 'Registration Form' and contains the following fields: 'First name', 'Last name', 'ID', 'Choose your ID type' (with a dropdown arrow), 'Date of birth' (with a date picker icon), and a placeholder 'dd/mm/yyyy'.

Elige su rol adecuado excepto el rol 'Client'. Ten en cuenta de que role 'Client' tiene su vista especial, es decir, limitada ya que se puede acceder solamente ciertas funcionalidades. Administrador, desarrollador comercial, desarrollador o diseñador tiene acceso a todas las páginas y los funcionalidades



The image shows a dropdown menu for selecting a role. The title is 'Role'. The menu is open, showing a list of roles with their descriptions. The first option is 'Choose your role' with a checkmark icon. The other options are: 'Administrator - Person responsible for carrying out the administrative support and general office', 'Business developer - Person works to improve an organization's market position and achieve', 'Developer - System, software, web or application developer', 'Designer - Web, graphic Designer', and 'Client - The recipient of the service who are willing to track thier project's progress'.

Una vez, Elige su rol de administrador, desarrollador comercial, desarrollador o diseñador, Company será deshabilitada ya que este field es para cliente y se puede elegir la oficina donde trabaja el usuario.

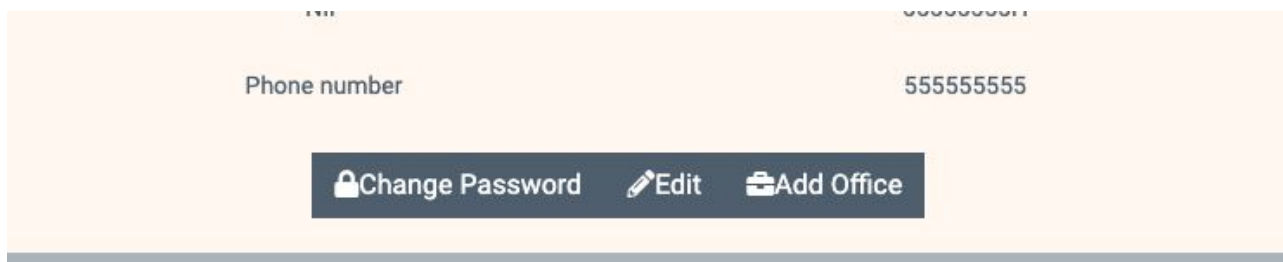


The image shows a registration form overlay on a wooden desk background. The form has two tabs: 'Login' and 'Register'. The 'Register' tab is active. The form fields are: 'Confirm Password' (text input), 'Role' (dropdown menu with 'Administrator - Person responsible for carrying out the adminis' selected), 'Company' (dropdown menu with 'Choose company where you work' selected), and 'Office' (dropdown menu with 'Choose office where you work' selected). There is a checkbox for 'Send me promotional offers' and a green 'Save' button at the bottom.

Si la oficina todavia no esta registrado previamente, el usuario puede dejarlo sin elegir y luego registra nueva oficina.



Para registrar oficina, haga clic en usuario en la barra de menú ubicada en la mano izquierda.

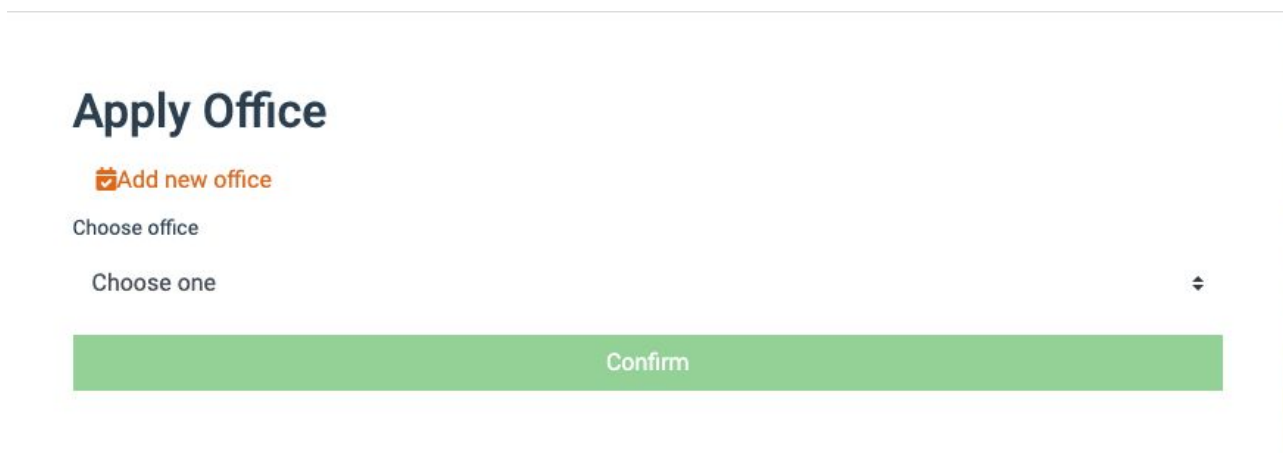


Phone number 555555555

 Change Password  Edit  Add Office

Luego, haga clic en el botón "Add office", puede elegir la oficina si ya existe o darle "Add new office" a insertar los datos de la oficina y haga clic en el botón "Confirm".

imagen de pagina office



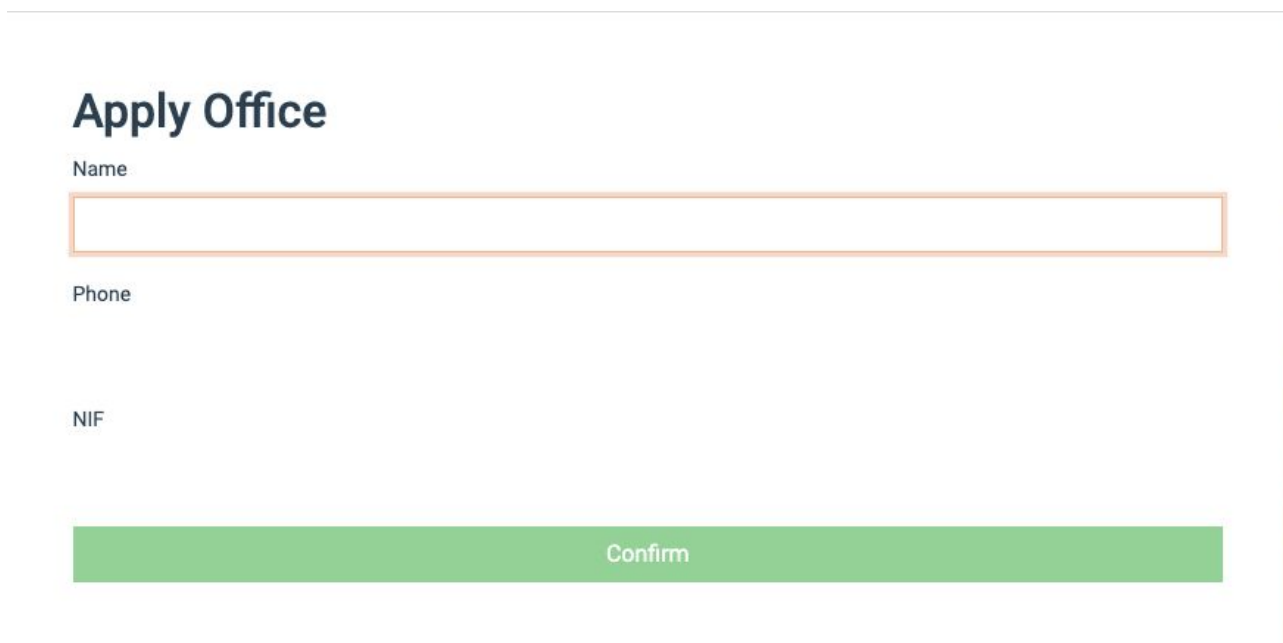
### Apply Office

 Add new office

Choose office

Choose one 

Confirm



### Apply Office

Name

Phone

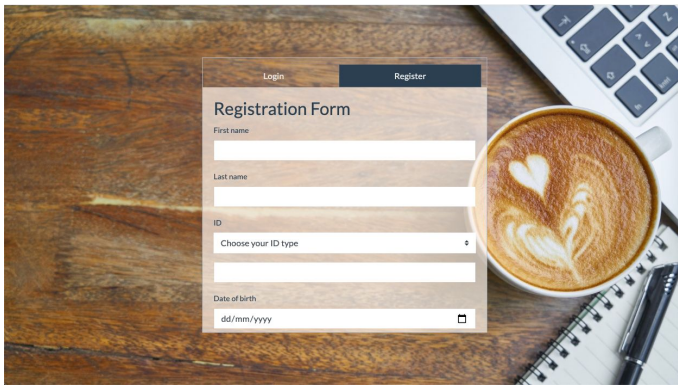
NIF

Confirm

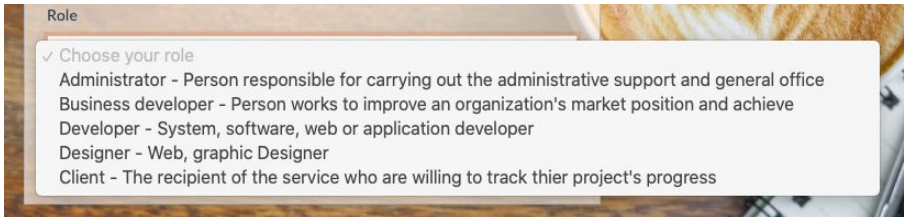
Configuración para Cliente

Haga clic en la pestaña de registro e inserte sus datos.





Asegúrese de elegir el rol 'Client'.



El cliente solo puede ver los detalles del proyecto y el progreso de ellos, lo que significa que puede verificar la tarea.

Una vez que tenga su cuenta, puede iniciar sesión la próxima vez, deberá insertar un nombre de usuario y contraseña para hacerlo, de lo contrario, no le permitirá entrar.



La vista de Empleados

Menu bar

La barra de menú contiene las siguientes funciones de forma desplegable. Proporciona acceso instantáneo a diferentes tareas y acciones.



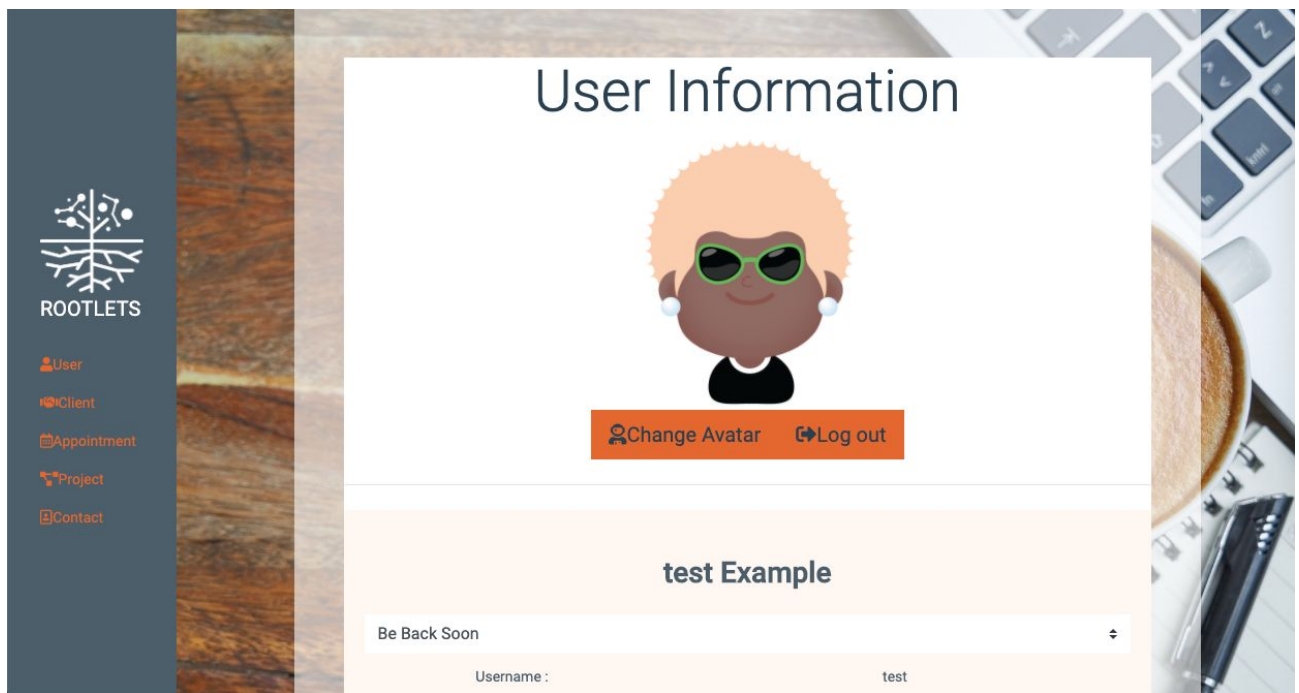
### 1. Logo

Haga clic en él y el usuario aterrizará en la página de inicio.

### 2. User

Haga clic en él y el usuario aterrizará en la página de información del usuario.





Puede cerrar sesión desde el botón "Log out" ubicado en la esquina superior derecha.

El botón "Change Password": Haga clic en él para mandar un correo electrónico que tenga un url que dirige a cambiar la contraseña.

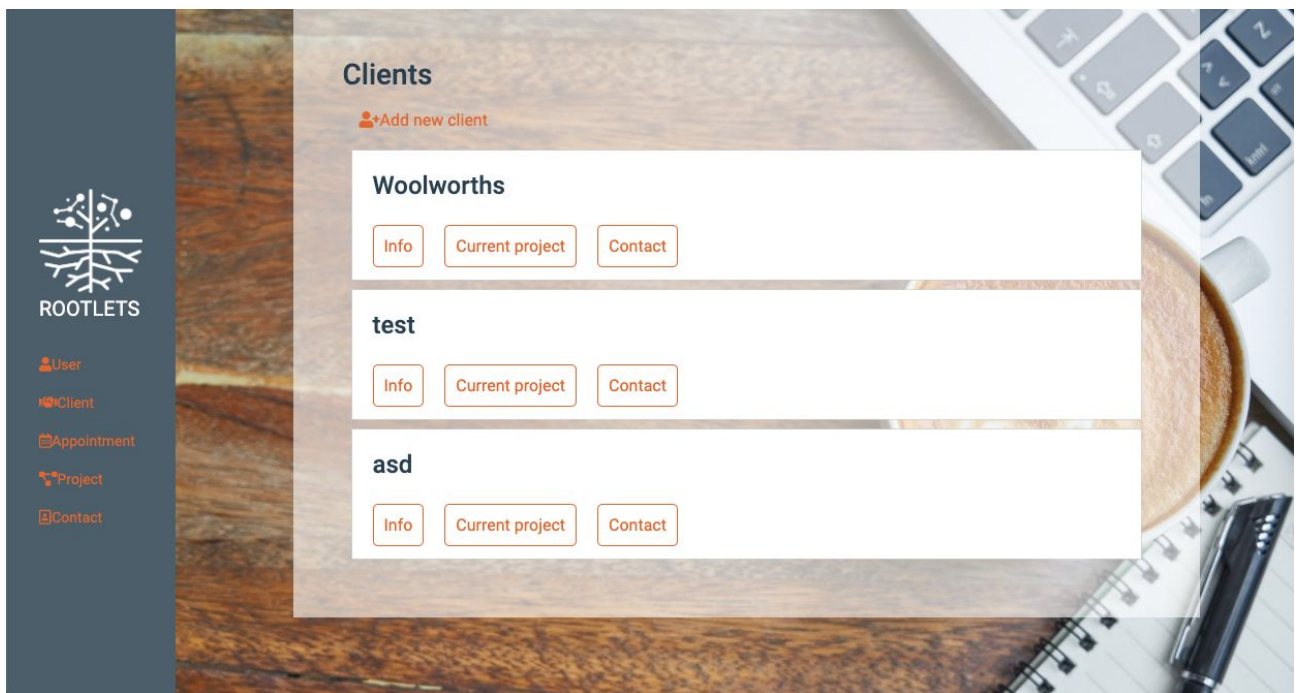
El botón "Edit": Haga clic en él para hacer cambios en sus detalles de usuario.

El botón "Add office": Agregar el oficina donde trabaja el usuario.

El botón "Change Avatar": Puede cambiar el avatar como desea el usuario.

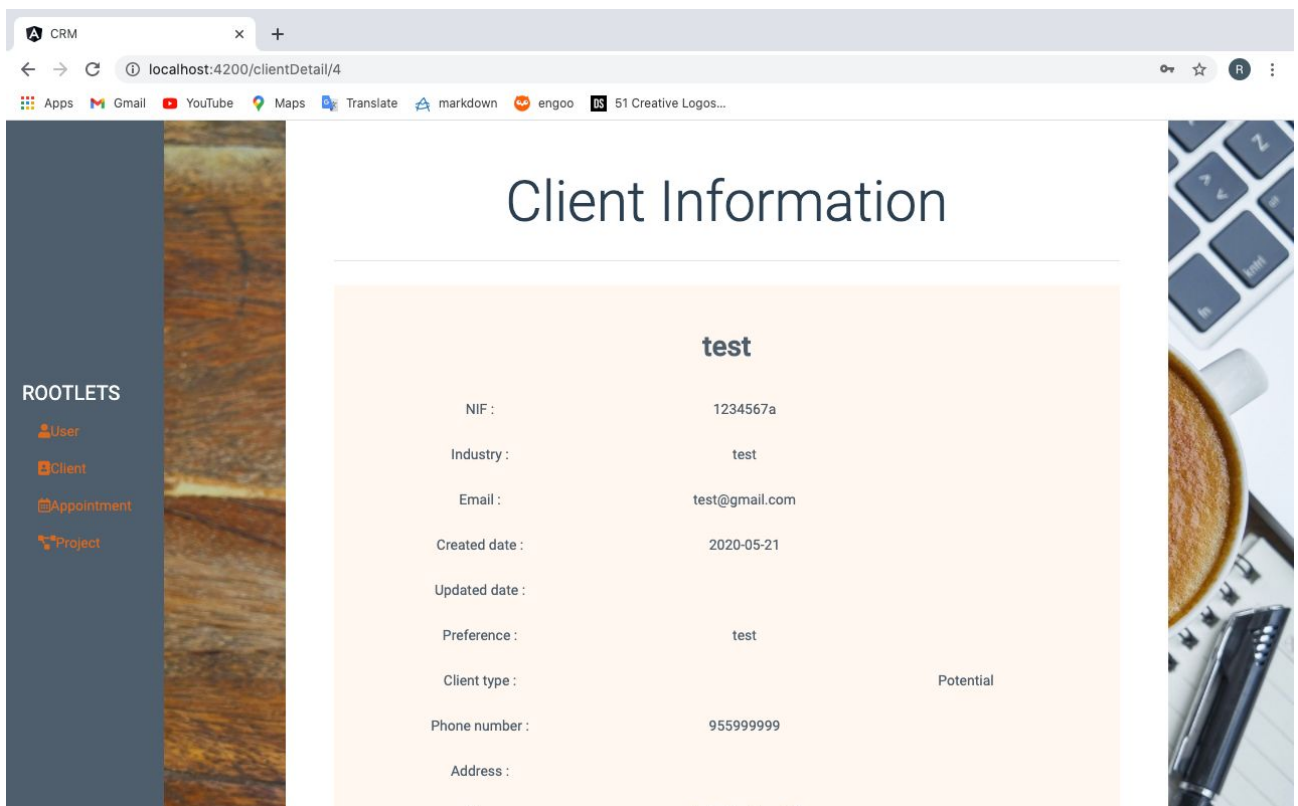
### 3. Client

Haga clic en él y el usuario aterrizará en la página de lista de clientes.



Muestra la lista de clientes existentes en el cuadro.

El botón "Info": Muestra más detalle de esa cliente.



El botón "Current project": Muestra proyecto con esa cliente si tiene.

El botón "Contact": Muestra lista de contacto de esa cliente, por ejemplo, información de marketing manager de empresa cliente.

El botón "Add new Client": Haga clic en él para agregar los datos del cliente.

### Add New Client

Company Name

NIF

Industry

Email

Preference

Client type

Phone [Add Phone Number](#)

#### 4. Appointment

Haga clic en él y el usuario aterrizará en la página de lista de meeting.

### Appointments

Today Upcoming Done

[Add new appointment](#)

**12:00 28-11-2020**  
First ever meeting with client B  
[Info](#)

**00:00 28-11-2020**  
testtest  
[Info](#)

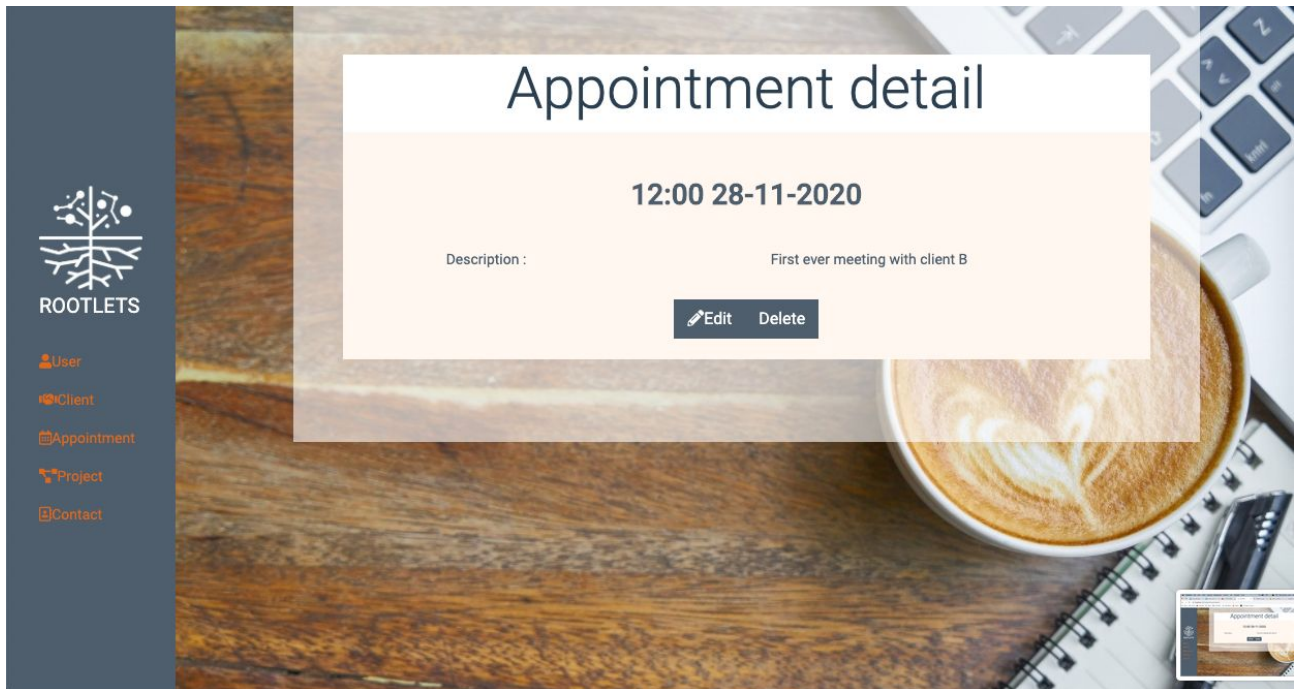
**17:35 08-12-2020**  
with new client  
[Info](#)

Muestra la lista de las citas que tenga.

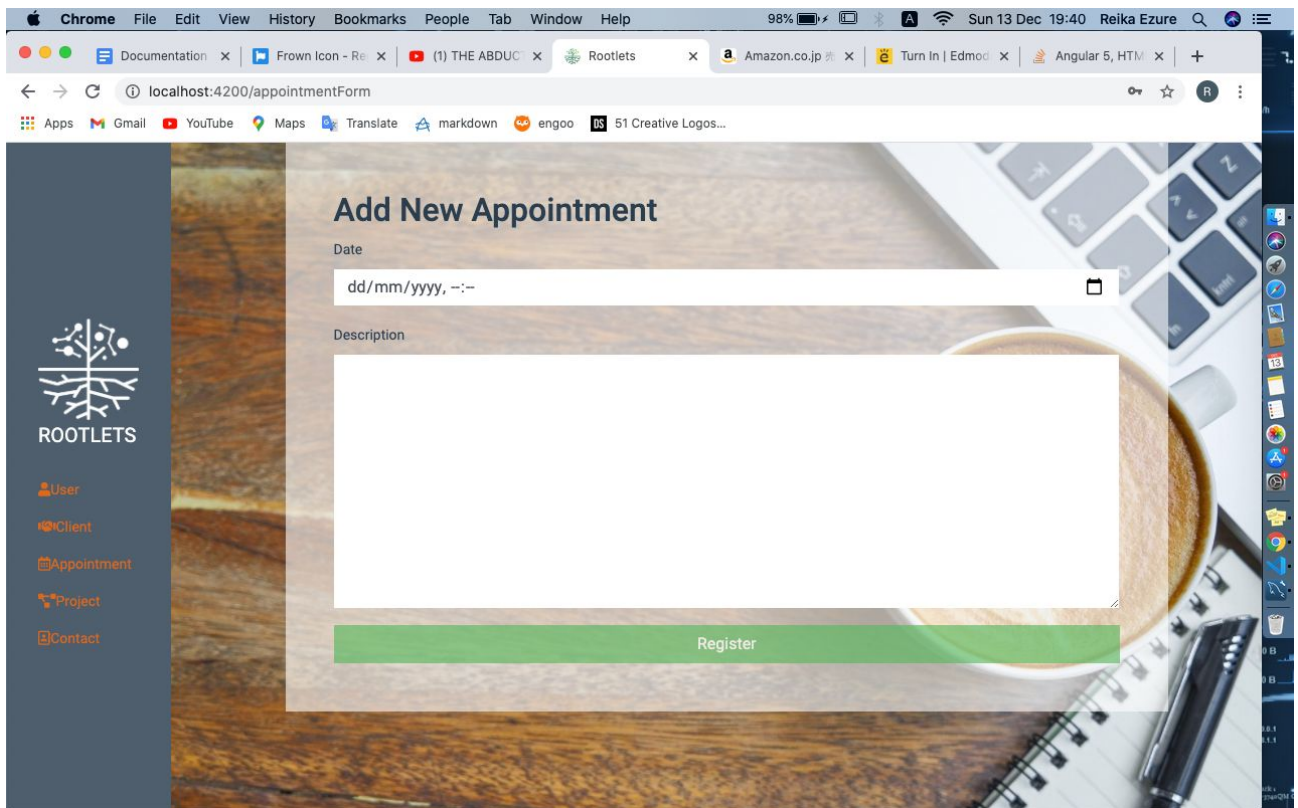
Puede filtrar la citas de hoy(Today), Proximos(Upcoming), Hecho(Done).



El botón "Info": Muestra más detalle de esa cita.

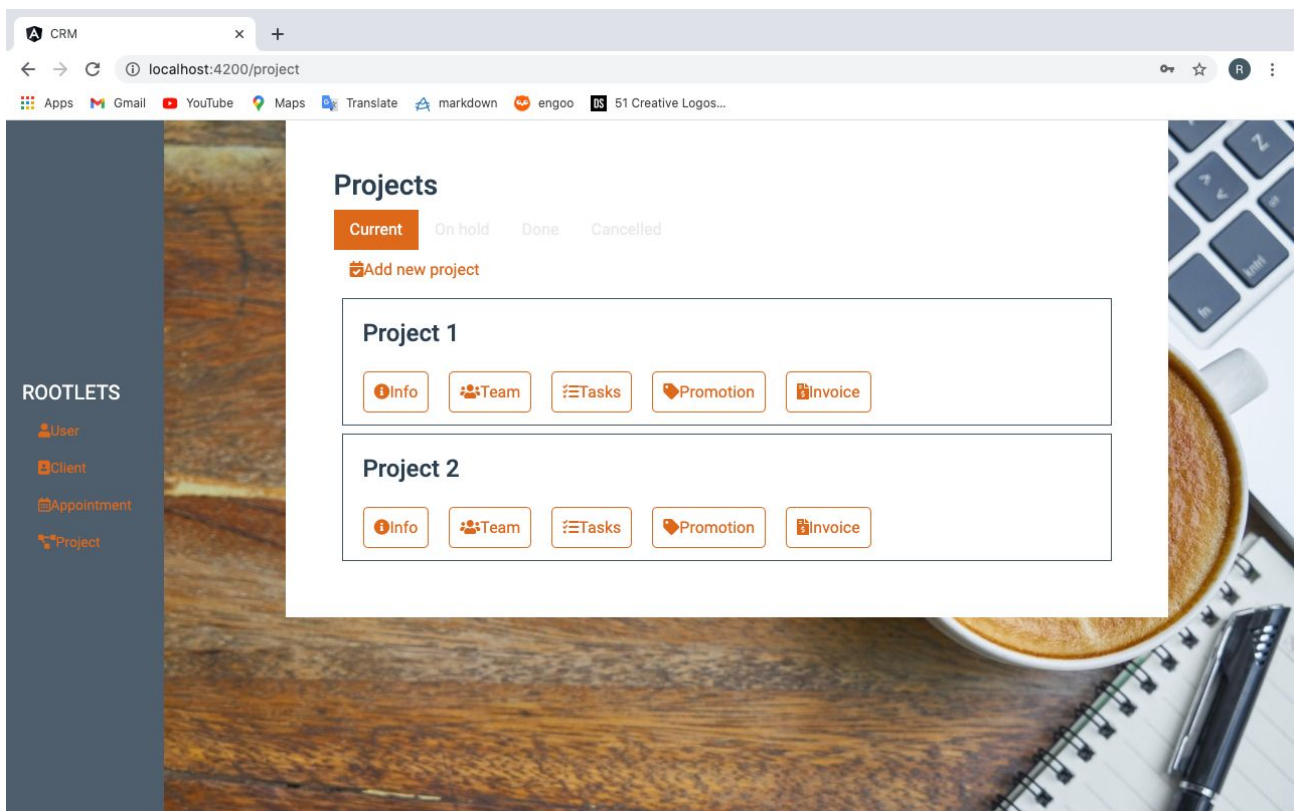


El botón "Add new": Haga clic en él para agregar nueva cita.



## 5. Project

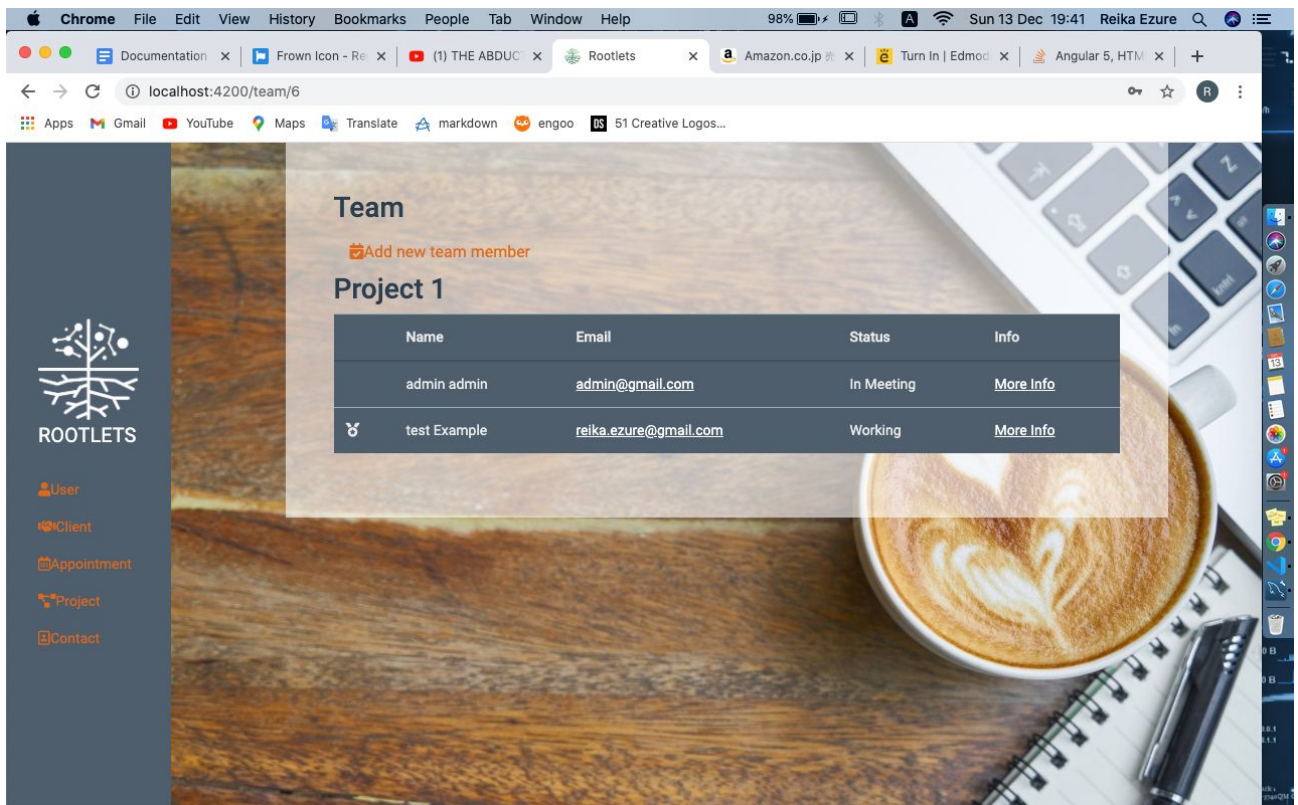
Haga clic en el menú "Project" para realizar cualquiera de las siguientes funciones.



Puede filtrar los proyectos según estado del proyecto, tanto como Actual(Current), A la espera(On hold), Hecho(Done), Cancelado(Cancelled)

Muestra lista de proyectos que ha asignados.

Team: Muestra miembro de su equipo con el que está trabajando actualmente.





Task: Muestra tareas que están planificadas para el proyecto. Cambia color de fila segun su estado.

In progress: Gris oscuro

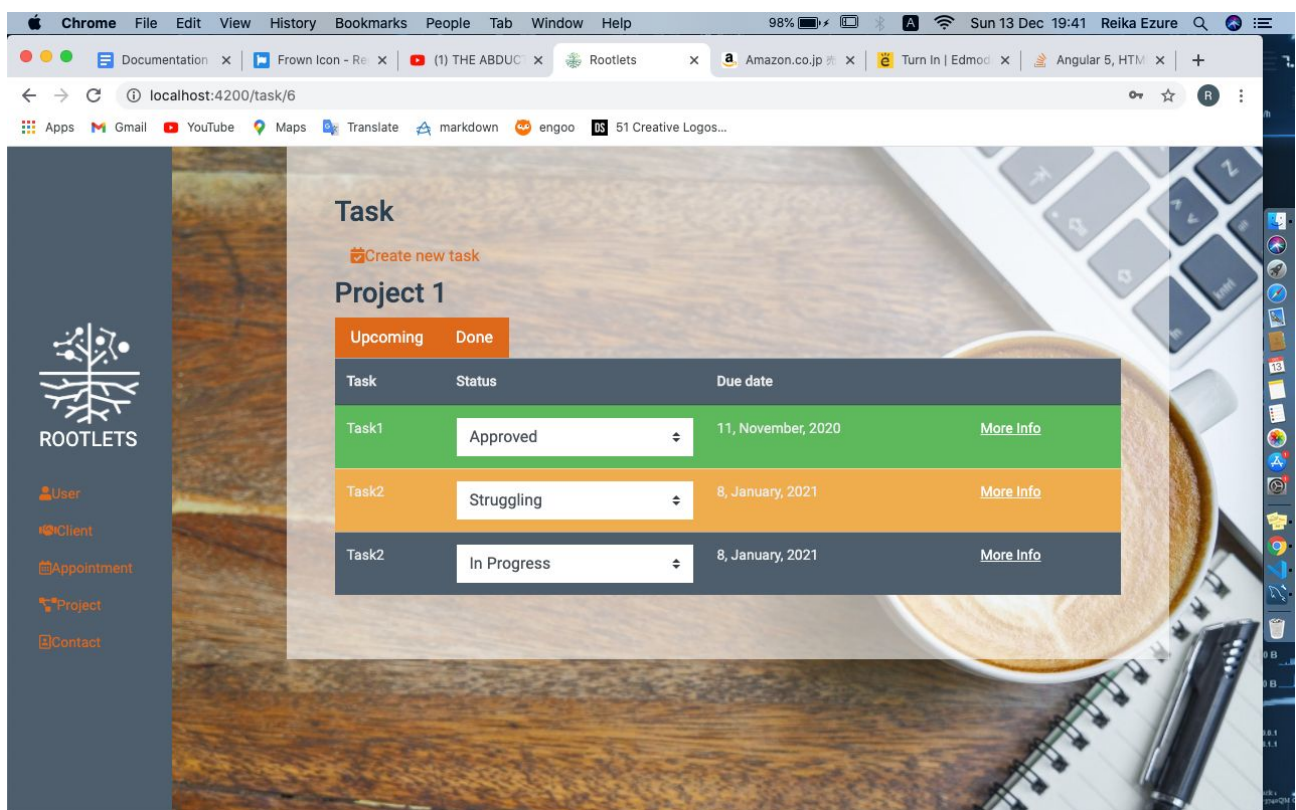
Done: Gris claro

Approved: Verde

Pending: Azul

Struggling: Amarillo

El estado de struggling va a aparecer como in progress a la vista del cliente así que los clientes no le va a dar la cuenta de que hay la tarea atacada.



Promotion: Muestra promoción aplicada a este proyecto. El administrador y el desarrollador de negocios pueden aplicar promociones al proyecto.





ROOTLETS

User

Client

## Apply promotion

[Add new promotion](#)

Choose promotion to apply to Project 1

Choose one

Confirm



ROOTLETS

User

Client

Appointment

Project

Contact

## Apply promotion

Name

OfferAmount

0

% OFF

Description

Confirm

Invoice: Imprima la factura del proyecto en formato pdf



ROOTLETS

User

Client

Appointment

Project

Contact

## Invoice detail

[Modify invoice](#)

### Project 2

|                |            |
|----------------|------------|
| Created Date : | 2020-05-05 |
| Issue Date :   | 2020-05-05 |
| IVA :          | 2100       |
| IRPF :         |            |
| NETO :         |            |
| Subtotal :     | 10000      |
| Total :        | 12100      |

Generate PDF

Si todavia no se ha registrado informacion de factura, dirige a formulario para que rellana.

**Invoice detail**

Create invoice

**Invoice**

IVA  
0

IRPF  
0

NETO  
0

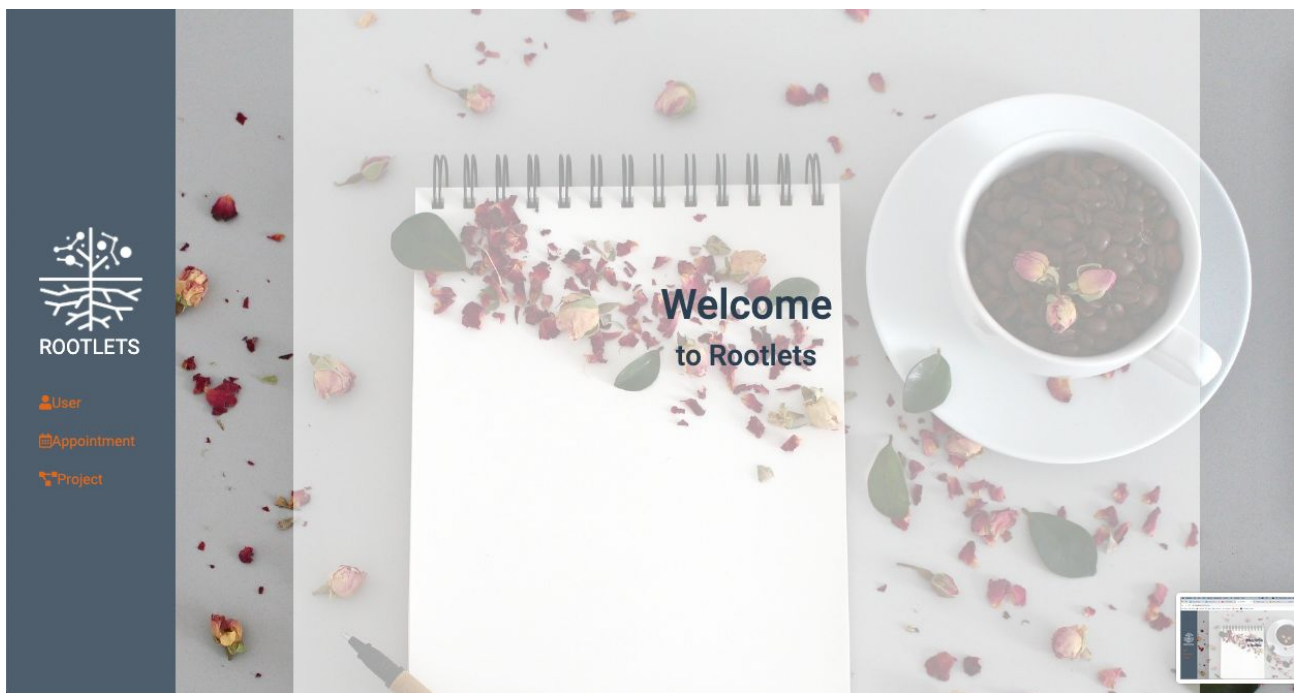
Subtotal  
0

Total  
0

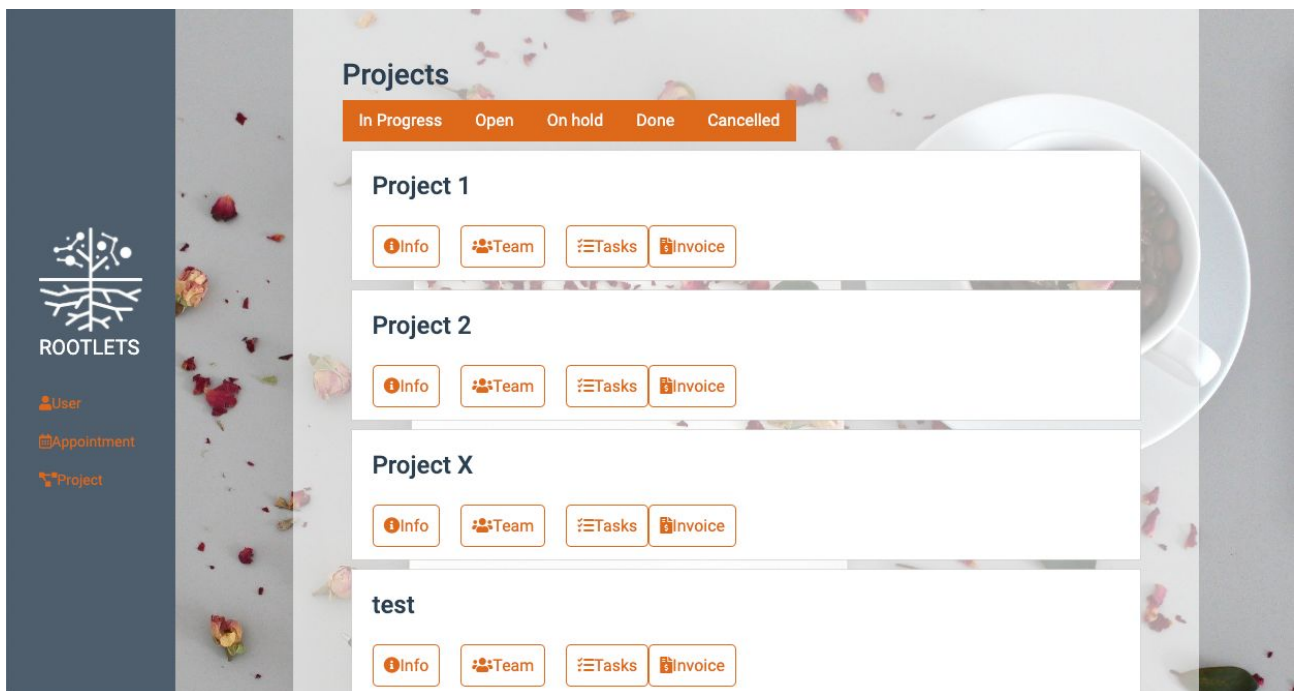
Confirm

### La vista de clientes

Los usuario 'Cliente' solo se puede acceder a la página de usuario, las citas y los proyectos. La página de usuario tanto como la página de citas son iguales con la vista de empleados.

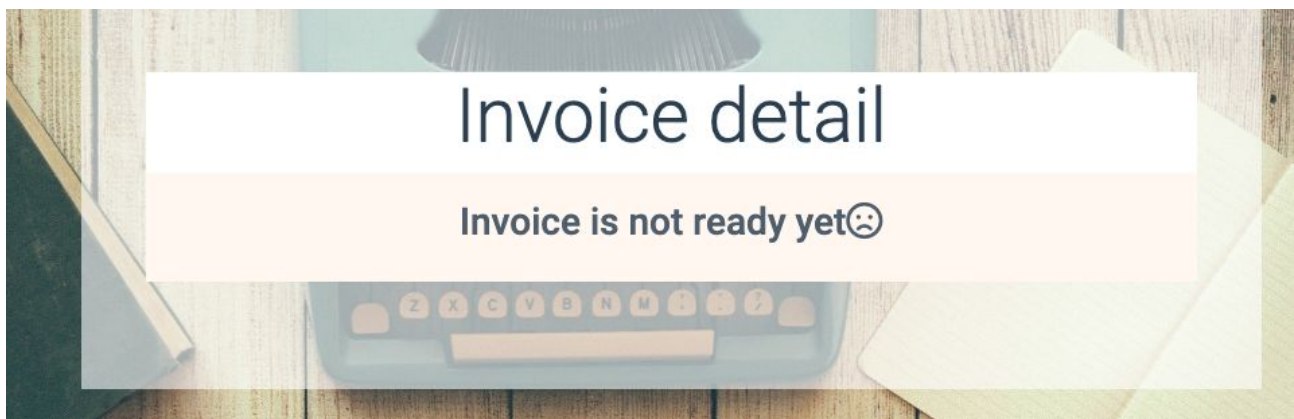


El cliente no puede añadir nuevo proyecto y modificarlo. Tampoco el cliente puede acceder a la página de promoción, sin embargo, se puede ver el detalle de la promoción aplicada a través de la página de información de cada proyecto.



La página de factura se puede acceder e imprimir a PDF siempre cuando hay factura registrada por empleados. Si no sale el mensaje que le diga 'no está disponible'.





## 9.- Bibliografía

Número de empresas en el sector TIC por ramas de actividad del sector TIC y periodo., 2020. *INE* [online],

- **Documentación de Angular 8**

Angular, 2020. *Angular.io* [online],

- **Documentación de Node js**

HTTP | Node.js v13.14.0 Documentation, 2020. *Nodejs.org* [online],

- **Documentación de Express web framework**

Express web framework (Node.js/JavaScript), 2020. *MDN Web Docs* [online],

- **Documentación de promise-mysql**

mysqljs/mysql, 2020. *GitHub* [online],

- **Documentación de Rest API usando node js y MySQL**

Node.js - RESTful API - Tutorialspoint, 2020. *Tutorialspoint.com* [online],

- **Documentación de Nodemailer**

REINMAN, ANDRIS, 2020, Nodemailer :: Nodemailer. *Nodemailer.com* [online]. 2020. [Accessed 18 June 2020]. Available from: <https://nodemailer.com/about/>,

- **Documentación de pdfmake**

pdfmake, 2020. *Pdfmake.github.io* [online],

- **Documentación de Ngx Cookie Service**

npm. 2020. *Ngx-Cookie-Service*. [online] Available at:

<<https://www.npmjs.com/package/ngx-cookie-service>> [Accessed 28 November 2020].

- **Documentación de bootstrap 4**

MARK OTTO, AND BOOTSTRAP CONTRIBUTORS, 2020, Introduction.

*Getbootstrap.com* [online]. 2020. [Accessed 18 June 2020]. Available from: <https://getbootstrap.com/docs/4.5/getting-started/introduction/>

# 10 - Anexo I.- Principales clases del código

## Modelos

```
Appointment{
  idAppointment?: number;
  date?: Date;
  description?: String;
  createdAt?: Date;
  updatedAt?: Date;
}

Client{
  personalityType?: number;
  clientCompany?: number;
}

ClientCompany{
  idClient?: number;
  companyName?: String;
  nif?: String;
  industry?: String;
  email?: String;
  createdAt?: Date;
  updatedAt?: Date;
  preference?: String;
  clientType_idClientType?: number;
  phone?: Array<Phone>;
  address?: Address;
  sns?: Array<Sns>;
}

Phone{
  client_idClient?: number;
  phoneNumber?: String;
}

Address{
  client_idClient?: number;
  addressLine?: String;
  city?: String;
  state?: String;
  country?: String;
  postalCode?: String;
}

Sns{
  client_idClient?: number;
  sns?: String;
  url?: String;
}

ClientType{
  idClientType?: number;
```



```
    type?: String;
    description?: String;
}
```

```
Employee{
    officeId?: number;
}
```

```
Invoice{
    idInvoice?: number;
    issueDate?: Date;
    createdDate?: Date;
    updatedDate?: Date;
    total?: number;
    subtotal?: number;
    neto?: number;
    iva?: number;
    irpf?: number;
    project_idProject?: number;
}
```

```
Login {
    idLogin?: number;
    username?: String;
    password?: String;
    email?: String;
}
```

```
Office{
    idOffice?: number;
    name?: String;
    phone?: String;
    nif?: String;
}
```

```
Project{
    idProject?: number;
    title?: String;
    description?: String;
    dueDate?: Date;
    createdDate?: Date;
    updatedDate?: Date;
    actualCompletionDate?: Date;
    budget?: number;
    price?: number;
    documentation?: String;
    feedback?: String;
    quantityOfChange?: number;
    status?: number;
    promold?: number;
    client_idClient?: number;
}
```

```
Promotion {
```

```

    idPromotion?: number;
    description?: string;
    offerAmount?: number;
    name?: string;
  }

  Task{
    idTask?: number;
    createdAt?: Date;
    description?: String;
    dueDate?: Date;
    actualCompletionDate?: Date;
    taskName?: String;
    status?: number;
    project_idProject?: number;
  }

  Team{
    idTeam?: number;
    project_idProject?: number;
    user_idUser?: number;
  }

  TeamLeader{
    teamLeaderId?: number;
    electedDate?: Date;
    updatedDate?: Date;
    team_idTeam?: number;
  }

  User{
    idUser?: number;
    firstName?: String;
    lastName?: String;
    birthDate?: Date;
    phone?: String;
    joinedDate?: Date;
    lastLogin?: Date;
    id?: String;
    idType?: String;
    avatar?;
    role?: number;
    status?: number;
    login_idLogin?:number;
  }

```

---

## Servicios

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Appointment } from '../models/Appointment';

```

```

@Injectable({
  providedIn: 'root'
})
export class AppointmentService {

  _url = 'http://localhost:3000';

  appointments : Appointment[];
  appointment : Appointment;

  constructor(private _http: HttpClient) { }

  getTodaysAppointments(){
    return this._http.get(`${this._url}/appointment`);
  }
  getUpcomingsAppointments(){
    return this._http.get(`${this._url}/appointment/upcoming`);
  }
  getDoneAppointments(){
    return this._http.get(`${this._url}/appointment/done`);
  }

  getAppointment(id: String | number){
    return this._http.get(`${this._url}/appointment/${id}`);
  }

  saveAppointment(appointment: Appointment){
    return this._http.post(`${this._url}/appointment`, appointment);
  }

  deleteAppointment(id: String | number){
    return this._http.delete(`${this._url}/appointment/${id}`);
  }

  updateAppointment(id: String | number, appointment: Appointment): Observable<Appointment>{
    return this._http.put(`${this._url}/appointment/${id}`, appointment);
  }
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

```

```

import { Login } from '../models/Login';

```

```

@Injectable({
  providedIn: 'root'
})
export class AuthenticationService {

```

```

  isLoggedIn: boolean = false;

```

```

loginDetail: Login;

_url = 'http://localhost:3000/login';

constructor( private _http: HttpClient ) { }

login(userData: any){
  return this._http.post(`${this._url}`, userData);
}

registerLogin(login: Login){
  return this._http.post(`${this._url}/register`, login);
}

getLogin(id: String | number){
  return this._http.get(`${this._url}/${id}`);
}

resetPassword(id: String | number, data: Login): Observable<Login>{
  return this._http.put(`${this._url}/reset/${id}`, data);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'

import { ClientCompany, Address, Phone, Sns } from '../models/ClientCompany';
import { Observable } from 'rxjs';
import { Client } from '../models/Client';

@Injectable({
  providedIn: 'root'
})
export class ClientService {

  _url = 'http://localhost:3000';

  clientCompanies : ClientCompany[];
  clientCompany : ClientCompany;
  phone : any; //Phone[];
  addr : Address;
  sns : any; //Sns[];

  constructor(private _http: HttpClient) { }

  getClients(){
    return this._http.get(`${this._url}/client`);
  }

  getClient(id: number){
    return this._http.get(`${this._url}/client/${id}`);
  }
}

```

```

saveClient(client: ClientCompany){
  return this._http.post(`${this._url}/client`, client);
}

deleteClient(id: number){
  return this._http.delete(`${this._url}/client/${id}`);
}

updateClient(id: String | number, client: ClientCompany): Observable<ClientCompany>{
  return this._http.put(`${this._url}/client/${id}`, client);
}

loadClientTypes(){
  return this._http.get(`${this._url}/clientType`);
}

saveAddress(address: Address){
  return this._http.post(`${this._url}/address`, address);
}

getAddress(id: number){
  return this._http.get(`${this._url}/address/${id}`);
}

savePhone(phone: Phone){
  return this._http.post(`${this._url}/phone`, phone);
}

getPhone(id: number){
  return this._http.get(`${this._url}/phone/${id}`);
}

saveSns(sns: Sns){
  return this._http.post(`${this._url}/sns`, sns);
}

getSns(id: number){
  return this._http.get(`${this._url}/sns/${id}`);
}
}

```

---

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { Office } from '../models/Office';

```

```

@Injectable({
  providedIn: 'root'
})
export class OfficeService {

```



```

_url = 'http://localhost:3000/office';

offices: Office[];
poffice: Office

constructor(private _http: HttpClient) { }

getOffices(){
  return this._http.get(`${this._url}`);
}

getOffice(id: number){
  return this._http.get(`${this._url}/${id}`);
}

saveOffice(office: Office){
  return this._http.post(`${this._url}`, office);
}

deleteOffice(id: number){
  return this._http.delete(`${this._url}/${id}`);
}

updateOffice(id: String | number, office: Office): Observable<Office>{
  return this._http.put(`${this._url}/client/${id}`, office);
}
}

```

---

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Address } from '../models/ClientCompany';
import pdfMake from 'pdfmake/build/pdfmake';
import pdfFonts from 'pdfmake/build/vfs_fonts';
import { Observable } from 'rxjs';
import { ClientCompany, Phone } from '../models/ClientCompany';
import { Invoice } from '../models/Invoice';
import { Office } from '../models/Office';
import { Project } from '../models/Project';
import { User } from '../models/User';

```

```

@Injectable({
  providedIn: 'root'
})
export class PdfService {
  pdfMake: any;

```

```

_url = 'http://localhost:3000/invoice';

```

```

invoices : Invoice[];
invoice : Invoice;

```

```

constructor(private _http: HttpClient) { }

getInvoices(){
  return this._http.get(`${this._url}`);
}

getInvoice(id: String | number){
  return this._http.get(`${this._url}/${id}`);
}

saveInvoice(invoice: Invoice){
  return this._http.post(`${this._url}`, invoice);
}

deleteInvoice(id: String | number){
  return this._http.delete(`${this._url}/${id}`);
}

updateInvoice(id: String | number, invoice: Invoice): Observable<Invoice>{
  return this._http.put(`${this._url}/${id}`, invoice);
}

//load the PDFMake library
async loadPdfMaker() {
  if (!this.pdfMake) {
    const pdfMakeModule = await import('pdfmake/build/pdfmake');
    const pdfFontsModule = await import('pdfmake/build/vfs_fonts');
    this.pdfMake = pdfMakeModule.default;
    this.pdfMake.vfs = pdfFontsModule.default.pdfMake.vfs;
  }
}

//generate pdf
async generatePdf(office: Office, user: User, client: ClientCompany, project: Project, invoice: Invoice,
phone: Phone[], address: Address) {
  await this.loadPdfMaker();
  const def = {
    content: [
      {
        style: 'tableExample',
        table: {
          body: [
            ['Invoice', 'Number: '+invoice.idInvoice],
            ['', 'Issue date: '+invoice.issueDate]
          ]
        },
        layout: 'noBorders'
      },
      {
        canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ],
        {
          style: 'tableExample',
          table: {
            body: [
              [client.companyName, "],

```

```

        [address.addressLine+' '+address.city+' '+address.state+' '+address.postalCode+' '+address.country, ""],
        ['NIF: '+client.nif, ""],
        ['Tel: '+phone[0].phoneNumber, 'Day of order: '+project.createdDate],
        ['Email: '+client.email, 'Day of submission: '+project.actualCompletionDate],
    ]
  },
  layout: 'noBorders'
},
{canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
'Concept',
{
  style: 'tableExample',
  table: {
    body: [
      ['Project title', project.title],
      ['Description', project.description]
    ]
  },
  layout: 'noBorders'
},
{canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
{
  style: 'tableExample',
  table: {
    body: [
      ['Subtotal', invoice.subtotal],
      ['IVA', invoice.iva],
      ['Total: ', invoice.total],
    ]
  },
  layout: 'noBorders'
},
{canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
office.name,
office.nif,
office.phone,
'Payment should be done within 15 days',
'Professional services that, having as their object their original artistic creations (illustrations), are provided to third parties by natural persons who are authors of paintings, drawings, engravings, lithographs, comic strips, comics or comics, as well as their essays and sketches and other original plastic works, whether or not they are applied.'
]
};
console.log("generating pdf...");
this.pdfMake.createPdf(def).open();
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

```

```

import { Project } from '../models/Project';

@Injectable({
  providedIn: 'root'
})
export class ProjectService {

  _url = 'http://localhost:3000';

  projectStatus: any;
  projects : Project[];
  project: Project;

  constructor(private _http: HttpClient) { }

  getProjects(){
    return this._http.get(`${this._url}/project`);
  }

  getProject(id: number){
    return this._http.get(`${this._url}/project/${id}`);
  }

  saveProject(project: Project){
    return this._http.post(`${this._url}/project`, project);
  }

  deleteProject(id: String | number){
    return this._http.delete(`${this._url}/project/${id}`);
  }

  updateProject(id: String | number, project: Project): Observable<Project>{
    return this._http.put(`${this._url}/project/${id}`, project);
  }

  loadProjectStatus(){
    return this._http.get(`${this._url}/projectStatus`);
  }
}

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Promotion } from '../models/Promotion';

@Injectable({
  providedIn: 'root'
})
export class PromotionService {

  promotions : Promotion[];
  promotion: Promotion;

```

```

_url = 'http://localhost:3000/promo';

constructor(private _http: HttpClient) { }

getPromotions(){
  return this._http.get(`${this._url}`);
}

getPromotion(id: number){
  return this._http.get(`${this._url}/${id}`);
}

savePromotion(promo: Promotion){
  return this._http.post(`${this._url}`, promo);
}

deletePromotion(id: String | number){
  return this._http.delete(`${this._url}/${id}`);
}

updatePromotion(id: String | number, promo: Promotion): Observable<Promotion>{
  return this._http.put(`${this._url}/${id}`, promo);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

```

```

import { Task } from '../models/Task';

```

```

@Injectable({
  providedIn: 'root'
})

```

```

export class TaskService {

```

```

  _url = 'http://localhost:3000';

```

```

  taskStatus: any;
  tasks: Task[];

```

```

constructor(private _http: HttpClient) { }

```

```

  getTasks(id: String | number){
    return this._http.get(`${this._url}/task/${id}`);
  }

```

```

  getTask(id: String | number){
    return this._http.get(`${this._url}/task/info/${id}`);
  }

```



```

saveTask(task: Task){
  return this._http.post(`${this._url}/task`, task);
}

deleteTask(id: String | number){
  return this._http.delete(`${this._url}/task/${id}`);
}

updateTask(id: String | number, task: Task): Observable<Task>{
  return this._http.put(`${this._url}/task/${id}`, task);
}

changeStatus(data: any){
  return this._http.post(`${this._url}/task/changeStatus`, data);
}
loadTaskStatus(){
  return this._http.get(`${this._url}/taskStatus`);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Team, TeamLeader } from '../models/Team';

@Injectable({
  providedIn: 'root'
})
export class TeamService {

  _url = 'http://localhost:3000';

  teams: Team[];

  constructor(private _http: HttpClient) { }

  getTeams(){
    return this._http.get(`${this._url}/team`);
  }

  getTeam(id: String | number){
    return this._http.get(`${this._url}/team/${id}`);
  }

  saveTeam(team: Team){
    return this._http.post(`${this._url}/team`, team);
  }

  deleteTeam(id: String | number){

```

```

    return this._http.delete(`${this._url}/team/${id}`);
  }

  updateTeam(id: String | number, team: Team): Observable<Team>{
    return this._http.put(`${this._url}/team/${id}`, team);
  }

  getTeamLeaders(){
    return this._http.get(`${this._url}/teamLeader`);
  }

  getTeamLeader(id: String | number){
    return this._http.get(`${this._url}/teamLeader/${id}`);
  }

  saveTeamLeader(teamLeader: TeamLeader){
    return this._http.post(`${this._url}/teamLeader`, teamLeader);
  }

  deleteTeamLeader(id: String | number){
    return this._http.delete(`${this._url}/teamLeader/${id}`);
  }

  updateTeamLeader(id: String | number, teamLeader: TeamLeader): Observable<TeamLeader>{
    return this._http.put(`${this._url}/teamLeader/${id}`, teamLeader);
  }
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'

import { Observable } from 'rxjs';

import { Login } from '../models/Login';
import { User, UserRole, UserStatus } from '../models/User';

@Injectable({
  providedIn: 'root'
})
export class UserService {

  _url = 'http://localhost:3000';

  loggedInUser: User;
  roles: any; //UserRole[]
  status: any; //UserStatus[]

  constructor(private _http: HttpClient) { }

  loadRoles(){
    return this._http.get(`${this._url}/userRole`);
  }

```

```

    }

    loadUserStatus(){
      return this._http.get(`${this._url}/userStatus`);
    }

    registerUser(user: User){
      return this._http.post(`${this._url}/user/register`, user);
    }

    getUser(id: number){
      return this._http.get(`${this._url}/user/${id}`);
    }

    getUserById(id){
      return this._http.get(`${this._url}/user/byId/${id}`);
    }

    getEmail(id){
      return this._http.get(`${this._url}/login/getEmail/${id}`);
    }

    updateUser(id: number | number, user: User): Observable<User>{
      return this._http.put(`${this._url}/user/${id}`, user);
    }

    sendmail(data) {
      return this._http.post(`${this._url}/sendmail`, data);
    }

    login(id: number){
      return this._http.get(`${this._url}/user/login/${id}`);
    }

    logout(id: number){
      return this._http.get(`${this._url}/user/logout/${id}`);
    }

  }

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Appointment } from '../models/Appointment';

@Injectable({
  providedIn: 'root'
})
export class AppointmentService {

```

```

_url = 'http://localhost:3000';

appointments : Appointment[];
appointment : Appointment;

constructor(private _http: HttpClient) { }

getTodaysAppointments(){
  return this._http.get(`${this._url}/appointment`);
}
getUpcomingsAppointments(){
  return this._http.get(`${this._url}/appointment/upcoming`);
}
getDoneAppointments(){
  return this._http.get(`${this._url}/appointment/done`);
}

getAppointment(id: String | number){
  return this._http.get(`${this._url}/appointment/${id}`);
}

saveAppointment(appointment: Appointment){
  return this._http.post(`${this._url}/appointment`, appointment);
}

deleteAppointment(id: String | number){
  return this._http.delete(`${this._url}/appointment/${id}`);
}

updateAppointment(id: String | number, appointment: Appointment): Observable<Appointment>{
  return this._http.put(`${this._url}/appointment/${id}`, appointment);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

```

```

import { Login } from '../models/Login';

```

```

@Injectable({
  providedIn: 'root'
})

```

```

export class AuthenticationService {

```

```

  isLoggedIn: boolean = false;
  loginDetail: Login;

```

```

_url = 'http://localhost:3000/login';

```

```

constructor( private _http: HttpClient ) { }

```

```

login(userData: any){
  return this._http.post(`${this._url}`, userData);
}

registerLogin(login: Login){
  return this._http.post(`${this._url}/register`, login);
}

getLogin(id: String | number){
  return this._http.get(`${this._url}/${id}`);
}

resetPassword(id: String | number, data: Login): Observable<Login>{
  return this._http.put(`${this._url}/reset/${id}`, data);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'

import { ClientCompany, Address, Phone, Sns } from '../models/ClientCompany';
import { Observable } from 'rxjs';
import { Client } from '../models/Client';

@Injectable({
  providedIn: 'root'
})
export class ClientService {

  _url = 'http://localhost:3000';

  clientCompanies : ClientCompany[];
  clientCompany : ClientCompany;
  phone : any; //Phone[];
  addr : Address;
  sns : any; //Sns[];

  constructor(private _http: HttpClient) { }

  getClients(){
    return this._http.get(`${this._url}/client`);
  }

  getClient(id: number){
    return this._http.get(`${this._url}/client/${id}`);
  }

  saveClient(client: ClientCompany){
    return this._http.post(`${this._url}/client`, client);
  }

  deleteClient(id: number){

```

```

    return this._http.delete(`${this._url}/client/${id}`);
  }

  updateClient(id: String | number, client: ClientCompany): Observable<ClientCompany>{
    return this._http.put(`${this._url}/client/${id}`, client);
  }

  loadClientTypes(){
    return this._http.get(`${this._url}/clientType`);
  }

  saveAddress(address: Address){
    return this._http.post(`${this._url}/address`, address);
  }

  getAddress(id: number){
    return this._http.get(`${this._url}/address/${id}`);
  }

  savePhone(phone: Phone){
    return this._http.post(`${this._url}/phone`, phone);
  }

  getPhone(id: number){
    return this._http.get(`${this._url}/phone/${id}`);
  }

  saveSns(sns: Sns){
    return this._http.post(`${this._url}/sns`, sns);
  }

  getSns(id: number){
    return this._http.get(`${this._url}/sns/${id}`);
  }
}

```

---

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { Office } from '../models/Office';

```

```

@Injectable({
  providedIn: 'root'
})
export class OfficeService {

```

```

  _url = 'http://localhost:3000/office';

```

```

  offices: Office[];
  pffice: Office

```

```

  constructor(private _http: HttpClient) { }

```



```

getOffices(){
  return this._http.get(`${this._url}`);
}

getOffice(id: number){
  return this._http.get(`${this._url}/${id}`);
}

saveOffice(office: Office){
  return this._http.post(`${this._url}`, office);
}

deleteOffice(id: number){
  return this._http.delete(`${this._url}/${id}`);
}

updateOffice(id: String | number, office: Office): Observable<Office>{
  return this._http.put(`${this._url}/client/${id}`, office);
}
}

```

---

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Address } from '../models/ClientCompany';
import pdfMake from 'pdfmake/build/pdfmake';
import pdfFonts from 'pdfmake/build/vfs_fonts';
import { Observable } from 'rxjs';
import { ClientCompany, Phone } from '../models/ClientCompany';
import { Invoice } from '../models/Invoice';
import { Office } from '../models/Office';
import { Project } from '../models/Project';
import { User } from '../models/User';

```

```

@Injectable({
  providedIn: 'root'
})
export class PdfService {
  pdfMake: any;

  _url = 'http://localhost:3000/invoice';

  invoices : Invoice[];
  invoice : Invoice;

  constructor(private _http: HttpClient) { }

  getInvoices(){
    return this._http.get(`${this._url}`);
  }
}

```

```

getInvoice(id: String | number){
  return this._http.get(`${this._url}/${id}`);
}

saveInvoice(invoice: Invoice){
  return this._http.post(`${this._url}`, invoice);
}

deleteInvoice(id: String | number){
  return this._http.delete(`${this._url}/${id}`);
}

updateInvoice(id: String | number, invoice: Invoice): Observable<Invoice>{
  return this._http.put(`${this._url}/${id}`, invoice);
}

//load the PDFMake library
async loadPdfMaker() {
  if (!this.pdfMake) {
    const pdfMakeModule = await import('pdfmake/build/pdfmake');
    const pdfFontsModule = await import('pdfmake/build/vfs_fonts');
    this.pdfMake = pdfMakeModule.default;
    this.pdfMake.vfs = pdfFontsModule.default.pdfMake.vfs;
  }
}

//generate pdf
async generatePdf(office: Office, user: User, client: ClientCompany, project: Project, invoice: Invoice,
phone: Phone[], address: Address) {
  await this.loadPdfMaker();
  const def = {
    content: [
      {
        style: 'tableExample',
        table: {
          body: [
            ['Invoice', 'Number: '+invoice.idInvoice],
            ['', 'Issue date: '+invoice.issueDate]
          ]
        },
        layout: 'noBorders'
      },
      {
        canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ],
        {
          style: 'tableExample',
          table: {
            body: [
              [client.companyName, ""],
              [address.addressLine+' '+address.city+' '+address.state+' '+address.postalCode+' '+address.country, ""],
              ['NIF: '+client.nif, ""],
              ['Tel: '+phone[0].phoneNumber, 'Day of order: '+project.createdDate],
              ['Email: '+client.email, 'Day of submission: '+project.actualCompletionDate],
            ]
          }
        }
      }
    ]
  }
}

```

```

    },
    layout: 'noBorders'
  },
  {canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
  'Concept',
  {
    style: 'tableExample',
    table: {
      body: [
        ['Project tittle', project.title],
        ['Description', project.description]
      ]
    },
    layout: 'noBorders'
  },
  {canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
  {
    style: 'tableExample',
    table: {
      body: [
        ['Subtotal', invoice.subtotal],
        ['IVA', invoice.iva],
        ['Total: ', invoice.total],
      ]
    },
    layout: 'noBorders'
  },
  {canvas: [ { type: 'line', x1: 0, y1: 0, x2: 515, y2: 0, lineWidth: 1 } ]},
  office.name,
  office.nif,
  office.phone,
  'Payment should be done within 15 days',
  'Professional services that, having as their object their original artistic creations (illustrations), are
provided to third parties by natural persons who are authors of paintings, drawings, engravings, lithographs,
comic strips, comics or comics, as well as their essays and sketches and other original plastic works,
whether or not they are applied.'
]
};
console.log("genarating pdf...");
this.pdfMake.createPdf(def).open();
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Project } from '../models/Project';

@Injectable({
  providedIn: 'root'
})

```

```

export class ProjectService {

  _url = 'http://localhost:3000';

  projectStatus: any;
  projects : Project[];
  project: Project;

  constructor(private _http: HttpClient) { }

  getProjects(){
    return this._http.get(`${this._url}/project`);
  }

  getProject(id: number){
    return this._http.get(`${this._url}/project/${id}`);
  }

  saveProject(project: Project){
    return this._http.post(`${this._url}/project`, project);
  }

  deleteProject(id: String | number){
    return this._http.delete(`${this._url}/project/${id}`);
  }

  updateProject(id: String | number, project: Project): Observable<Project>{
    return this._http.put(`${this._url}/project/${id}`, project);
  }

  loadProjectStatus(){
    return this._http.get(`${this._url}/projectStatus`);
  }
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

```

```

import { Promotion } from '../models/Promotion';

```

```

@Injectable({
  providedIn: 'root'
})
export class PromotionService {

```

```

  promotions : Promotion[];
  promotion: Promotion;

```

```

  _url = 'http://localhost:3000/promo';

```

```

  constructor(private _http: HttpClient) { }

```

```

getPromotions(){
  return this._http.get(`${this._url}`);
}

getPromotion(id: number){
  return this._http.get(`${this._url}/${id}`);
}

savePromotion(promo: Promotion){
  return this._http.post(`${this._url}`, promo);
}

deletePromotion(id: String | number){
  return this._http.delete(`${this._url}/${id}`);
}

updatePromotion(id: String | number, promo: Promotion): Observable<Promotion>{
  return this._http.put(`${this._url}/${id}`, promo);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

```

```

import { Task } from '../models/Task';

```

```

@Injectable({
  providedIn: 'root'
})

```

```

export class TaskService {

```

```

  _url = 'http://localhost:3000';

```

```

  taskStatus: any;
  tasks: Task[];

```

```

  constructor(private _http: HttpClient) { }

```

```

  getTasks(id: String | number){
    return this._http.get(`${this._url}/task/${id}`);
  }

```

```

  getTask(id: String | number){
    return this._http.get(`${this._url}/task/info/${id}`);
  }

```

```

  saveTask(task: Task){
    return this._http.post(`${this._url}/task`, task);
  }

```

```

deleteTask(id: String | number){
  return this._http.delete(`${this._url}/task/${id}`);
}

updateTask(id: String | number, task: Task): Observable<Task>{
  return this._http.put(`${this._url}/task/${id}`, task);
}

changeStatus(data: any){
  return this._http.post(`${this._url}/task/changeStatus`, data);
}
loadTaskStatus(){
  return this._http.get(`${this._url}/taskStatus`);
}
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'
import { Observable } from 'rxjs';

import { Team, TeamLeader } from '../models/Team';

@Injectable({
  providedIn: 'root'
})
export class TeamService {

  _url = 'http://localhost:3000';

  teams: Team[];

  constructor(private _http: HttpClient) { }

  getTeams(){
    return this._http.get(`${this._url}/team`);
  }

  getTeam(id: String | number){
    return this._http.get(`${this._url}/team/${id}`);
  }

  saveTeam(team: Team){
    return this._http.post(`${this._url}/team`, team);
  }

  deleteTeam(id: String | number){
    return this._http.delete(`${this._url}/team/${id}`);
  }

  updateTeam(id: String | number, team: Team): Observable<Team>{

```



```

    return this._http.put(`${this._url}/team/${id}`, team);
  }

  getTeamLeaders(){
    return this._http.get(`${this._url}/teamLeader`);
  }

  getTeamLeader(id: String | number){
    return this._http.get(`${this._url}/teamLeader/${id}`);
  }

  saveTeamLeader(teamLeader: TeamLeader){
    return this._http.post(`${this._url}/teamLeader`, teamLeader);
  }

  deleteTeamLeader(id: String | number){
    return this._http.delete(`${this._url}/teamLeader/${id}`);
  }

  updateTeamLeader(id: String | number, teamLeader: TeamLeader): Observable<TeamLeader>{
    return this._http.put(`${this._url}/teamLeader/${id}`, teamLeader);
  }
}

```

---

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http'

import { Observable } from 'rxjs';

import { Login } from '../models/Login';
import { User, UserRole, UserStatus } from '../models/User';

@Injectable({
  providedIn: 'root'
})
export class UserService {

  _url = 'http://localhost:3000';

  loggedInUser: User;
  roles: any; //UserRole[]
  status: any; //UserStatus[]

  constructor(private _http: HttpClient) { }

  loadRoles(){
    return this._http.get(`${this._url}/userRole`);
  }

  loadUserStatus(){
    return this._http.get(`${this._url}/userStatus`);
  }

```

```

}

registerUser(user: User){
  return this._http.post(`${this._url}/user/register`, user);
}

getUser(id: number){
  return this._http.get(`${this._url}/user/${id}`);
}

getUserById(id){
  return this._http.get(`${this._url}/user/byId/${id}`);
}

getEmail(id){
  return this._http.get(`${this._url}/login/getEmail/${id}`);
}

updateUser(id: number | number, user: User): Observable<User>{
  return this._http.put(`${this._url}/user/${id}`, user);
}

sendmail(data) {
  return this._http.post(`${this._url}/sendmail`, data);
}

login(id: number){
  return this._http.get(`${this._url}/user/login/${id}`);
}

logout(id: number){
  return this._http.get(`${this._url}/user/logout/${id}`);
}

}

```

## 11 - Anexo 2: Opinión personal sobre la FCT

Estoy bastante contenta con la práctica ya que me tratan como empleado y me dejan ser parte del proyecto real. Disfruto enfrentando desafíos, así que no me gustaría que me trataran como estudiante y yo hiciera algún proyecto falso. Estoy trabajando en parte de diseño como la creación de logotipo e isotipo de aplicación, diseño de banner, diseño estacionario y diseño de interfaz. Es cierto que realmente no estoy tocando mucho código, sin embargo estoy aprendiendo estar en un proyecto real en una empresa real y tengo la responsabilidad de mi trabajo. Además, mi trabajo dará la primera impresión a los usuarios y clientes, por lo que hay suficiente presión y responsabilidad.

También utilizo la vista de código en REACT, de la que realmente no tengo mucha idea, pero me enseñan muy bien con paciencia, así que me siento muy afortunado de estar en este entorno laboral.

